

SOFTWARE DEVELOPMENT *WITHOUT* CHAOS

A Framework for Making Strategic
Decisions & Achieving Predictable Software
Without Being Technical



PABLO LEONE

SOFTWARE DEVELOPMENT WITHOUT CHAOS

A Framework for Making Strategic Decisions &
Achieving Predictable Software Without Being Technical

Copyright © 2025 Pablo Leone. All rights reserved.

This book is licensed under the [Creative Commons Attribution-NoDerivatives 4.0 International License](https://creativecommons.org/licenses/by-nd/4.0/).

You are **free to share, translate, and redistribute** this work — even for commercial purposes — as long as you credit Pablo Leone as the original author and do not modify or remix the content in a way that distorts its original meaning.

Please link to the official version or cite the author when sharing publicly.

Official site: <https://www.linkedin.com/in/pabloleone>

To my wife and son, who have given me the time and freedom to gather the experience behind these pages. This book is my way of leaving something tangible for a better world—proving that challenges we often accept as “unsolvable” actually have answers.

– *Pablo Leone*

Contents

[Introduction](#)

[What Is the SDLC and Why Is It Important?](#)

[The SDLC as a Machine: Input, Process, and Output](#)

[The Black Box of Software Development: How Ideas Turn into Products](#)

[Stories](#)

[Story 1: When Development Defines the Product](#)

[Story 2: The Billing Company and Directionless Development](#)

[Story 3: The Dissonance Between Team Structure and Project Goals](#)

[Story 4: Building a System Unknowingly](#)

[Story 5: The Hybrid That Didn't Work](#)

[Story 6: The Dual-Track Success](#)

[Final Note on Project Risk](#)

[Chapter 1: The Software Development Machine](#)

[1.1 The SDLC as a System of Nested Components](#)

[1.2 Types of Projects in the SDLC](#)

[1.2.1 Type A Projects: Experimental or MVPs](#)

[1.2.2 Type B Projects: Scalable Systems](#)

[1.3 Configuring the Machine for Each Type of Project](#)

[1.4 Working Backwards from the Desired Output](#)

[Chapter 2: Types of Projects](#)

[2.1 Type A Projects: Experimental / MVPs](#)

[2.2 Type B Projects: Systems / Scalable](#)

[2.3 Key Differences Between Type A and Type B Projects](#)

[2.4 How to Choose the Right Approach](#)

[2.5 Why a Hybrid Approach Is a Bad Idea](#)

[Chapter 3: Discovery Meets Delivery](#)

[3.1 Key Stages of the SDLC](#)

[3.2 Product Discovery: Exploration, Validation, and Adjustment](#)

[3.3 Software Delivery: Implementation, Scalability, and Optimization](#)

[3.4 How Iteration Between These Two Phases Refines Development](#)

[3.4.1 Beyond Sequential vs. Parallel](#)

[3.4.2 One Continuous Loop, Two Shifts in Mindset](#)

[3.5 Managing Type A and Type B Projects—Transition Considerations](#)

[Chapter 4: The Team Orchestration and Structure](#)

[4.1 Key Roles in Product Discovery vs. Software Delivery](#)

[4.2 Strategic Roles and Their Positioning in Different Projects](#)

[4.3 Differences in Team Configuration Based on Project Type](#)

[4.4 How to Structure Teams for Product Discovery and Software Delivery](#)

[4.4.1 Example: Two Team Configurations](#)

[4.5 Participation by Role in Each Phase \(Type A or B\)](#)

[4.6 Addressing Dissonance from Story 3 & Story 5](#)

[Chapter 5: Coordinating Through Communication](#)

[5.1 No Handoffs. Just Collaboration](#)

[5.2 Not All Discoveries Lead to Software Delivery](#)

[5.3 How to Avoid Errors in Requirements Specification](#)

[5.4 The Importance of Clarity in Problem Definition](#)

[5.5 Common Inefficiencies, Bad Practices, and How to Fix Them](#)

[Chapter 6: Project Management and Decision-Making](#)

[6.1 Why Project Management Matters](#)

[6.2 The Backlog: Not a Dumping Ground](#)

[6.3 ROI and Technical Debt: The Cost Grows Exponentially](#)

[6.4 Scrum vs. Kanban: Keep It Simple](#)

[6.5 Risk Management in Software Development](#)

[6.6 Shortcuts vs. Starting Small](#)

[Chapter 7: The Step-by-Step Software Development Process](#)

[7.1 From Ticket Creation to Final Delivery](#)

[7.1.1 Starting with Five Essential Questions](#)

[7.1.2 Step-by-Step Flow](#)

[7.2 Flowchart, Sequence Diagram, and Infrastructure Diagram](#)

[7.3 Ensuring Software Quality: A Strategic Perspective](#)

[7.4 Scaling the Development Team](#)

[7.5 Cultural Shifts: Transitioning from Type A to Type B](#)

[7.5.1 Why the Cultural Clash Occurs](#)

[7.5.2 Impact on the Development Team](#)

[7.5.3 Impact on Product & Business Teams](#)

[7.5.4 Navigating the Cultural Transition](#)

[7.5.5 Addressing Skill Atrophy When Transitioning from Type A to Type B](#)

[From Theory to Action](#)

[What We've Learned](#)

[Applying the Framework: Key Questions for CEOs & Founders](#)

[A Flexible Framework, Not a Rigid Recipe](#)

[Self-Assessment: Where Is Your SDLC Today?](#)

[Next Steps: Let's Work Together to Optimize Your SDLC](#)

[About the Author](#)

[Further Reading & Resources](#)

Share This Book

Help spread the ideas from *Software Development Without Chaos* by sharing it with other professionals looking to optimize their development processes.

The suggested hashtag for this book is **#SDWC**.

Find out what others are saying about it by searching for this hashtag on the platforms where you engage with your professional network.

Acknowledgments

My deepest thanks to Steve Asser for granting me my first opportunity to work independently and engage closely with clients. I'm equally grateful for my time in England, where countless experiences shaped the professional I am today. Special appreciation goes to Brian Tracy, whose Phoenix seminar helped me recognize that I hold the reins of my life and that my actions truly matter; his influence compelled me to write this book. Finally, I owe a heartfelt mention to Pete Buckney, creator of the OOPSI framework. His trust, clarity, and unrestrained guidance were pivotal in structuring these ideas and bringing them to life.

Introduction

This book is designed to offer a clear and practical overview of how to effectively manage the Software Development Lifecycle (SDLC). Throughout its chapters, you will see how each phase of the SDLC connects with agile methodologies (*approaches focused on flexibility, rapid feedback, and delivering value incrementally*), references to renowned industry books, and real-world examples that will help you improve both the quality and speed of your software deliveries. Above all, you will find analogies and visuals intended to make even the most technical details accessible for CEOs or founders without a software background.

Can you imagine developing software with clarity, speed, and tangible results, without getting lost in technical complexities or confusing methodologies? This book provides exactly that: a practical, definitive guide to effectively managing the Software Development Life Cycle (SDLC), aligned with current industry standards and proven techniques.

Here, you'll discover three key elements:

1. The SDLC as a Configurable Machine

You'll learn how to structure your development process—from agile MVPs (*Minimum Viable Products, basic versions built quickly to test a core business idea with real users*) to robust, scalable systems (*software designed to handle growth in users and data reliably*)—by configuring your team, tools, and architecture (*the fundamental design and structure of the software, affecting its stability and flexibility*) to meet exactly what your business needs. This proven approach ensures rapid deliveries without compromising quality, eliminating hidden costs and unnecessary frustrations.

2. Mastering the Product Discovery Phase

The crucial step of ensuring you're building the right product before investing heavily in development. Before writing a single line of code, clearly validating the problem you're solving and understanding its impact on users and your business is crucial. You'll learn how to transform ideas into concrete, validated specifications (*detailed requirements or blueprints*) that your team can implement immediately, preventing resource waste on developments that deliver no real value.

3. Real-World Lessons

At the beginning, you'll find real stories of companies that faced critical issues—such as delivery delays, uncontrolled costs, or difficult-to-maintain products—due to lacking a clear SDLC structure. But you'll also see how these issues could have been easily avoided by applying the practical approach presented in this book.

Why trust this guide?

Because I've personally experienced development challenges across various environments and company sizes, learning firsthand what truly works when pressure rises and expectations aren't always clear. This book consolidates those lessons into a practical method you can directly apply, without needing to search for additional answers across dozens of different books.

If you're a CEO or founder, you'll quickly gain a strategic view on turning your ideas into tangible, scalable, and profitable products, removing the "black box" from development. With this guide, you can make informed decisions, align your technical teams, and ensure every software effort generates tangible value.

Welcome to a new level of effectiveness and control in software development. Let's get started!

What Is the SDLC and Why Is It Important?

To understand the SDLC, imagine running an **industrial kitchen** (or a car factory if you prefer):

- When it's well-configured—chefs (*developers*), sous-chefs (*like Quality Assurance or QA, ensuring the software works correctly and DevOps focused on streamlining development and reliable operations*), and equipment (*tools/methodologies*) all in place—you can serve dishes (*software features*) on time and with consistent quality.
- If it's poorly configured, ingredients get wasted, dishes come out late or incorrectly, and you risk frustrating both the team and your customers.

The **Software Development Lifecycle (SDLC)** is this “kitchen” for software. It defines how a software product is conceived, developed, and maintained. Done right, it allows you to deliver high-quality products efficiently. But when the SDLC is poorly structured, it creates delays, wastes resources, and generates frustration across both technical teams and business stakeholders.

Note: *A well-defined SDLC is not unnecessary bureaucracy—it's your main tool to ensure that technology investments translate into predictable, business-aligned results, avoiding hidden costs and frustration.*

A well-designed SDLC provides:

- Greater predictability in delivery times (*fewer “late dishes”*).
- Better alignment between development and business (*everyone agrees on the “menu”*).
- Less wasted effort and unnecessary costs (*you’re not prepping ingredients no one ordered*).
- Higher overall software quality (*like serving dishes that consistently taste great and meet customer expectations*).

Measuring **success** in your SDLC can involve several **KPIs** (Key Performance Indicators), such as:

- **Time to market** (*how quickly you can release new features → how fast dishes leave the kitchen and reach the table*),
 - **Quality metrics** (*how many defects appear in production → how often dishes come back because something’s wrong*),
 - **ROI** (*Return on Investment → the value delivered compared to the cost of development → whether the dish earns more than it costs to prepare*),
 - **User satisfaction** (*direct feedback or adoption rates → if customers leave happy, come back, and recommend your restaurant*).
-

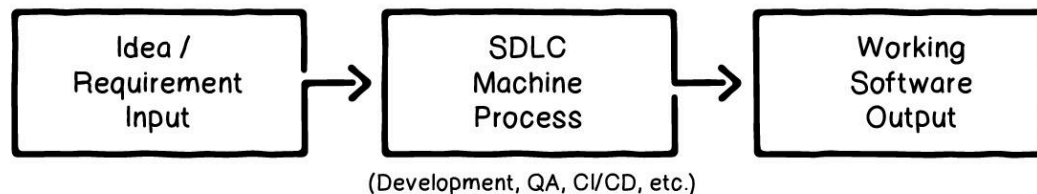
The SDLC as a Machine: Input, Process, and Output

To visualize the SDLC more concretely, think of it as a machine on a production line, or as an industrial kitchen with defined stations—just like in our earlier analogy:

1. **Input** – The original idea: a user need, a business opportunity, or any problem you want to solve.
2. **Process** – The “cooking” phase: your development team uses tools, frameworks (*reusable code structures or guides that simplify and speed up development*), and methodologies (*like Agile, Scrum, or Kanban—approaches that help teams manage work flexibly and deliver value iteratively*) to transform the idea into functional software.

3. **Output** – The “final dish”: an MVP, a beta release (*a pre-launch version for testing with a limited group of users*), or a fully scalable system.

Below is a simple conceptual diagram you might imagine for this “machine” (*or kitchen line*):



- The box in the middle represents the SDLC: all your development activities, coordination, testing, and deployment.
- “CI/CD” (*Continuous Integration/Continuous Delivery*) refers to automated processes that help teams deliver software updates faster and more reliably by integrating and deploying code changes frequently, reducing manual errors and delays.

If this machine (*or kitchen*) isn’t well configured, you get unpredictable results:

- Software may take longer than expected,
- You might accumulate **technical debt**

Note: *Technical debt is like the “mess” you leave behind when rushing to cook during a busy shift—it gets the job done, but cleaning it up later takes time and effort. In software, it refers to the future cost of quick or suboptimal solutions that must be refactored (cleaned up or restructured internally) to ensure long-term stability. This isn’t just a technical issue—it directly impacts your budget, your delivery speed, and your team’s agility. While small amounts of debt may be acceptable in early-stage experiments, letting it grow unchecked in long-term systems is a strategic mistake you must actively manage.*

- Or worst of all, you end up with a product that fails to meet business expectations.

When set up correctly, however, you can monitor key indicators—like on-time delivery, defect rates, or positive user feedback—to confirm that your “machine” is running smoothly.

The Black Box of Software Development: How Ideas Turn into Products

One of the biggest challenges for non-technical leaders is that development often feels like a “**black box**.” Ideas go in; at some unknown point, software comes out. But what happens in between?

Without a clear framework (*or a well-defined SDLC*), that black box can become chaotic:

- Requirements or ideas enter without a clear process.
- The development team may lack strategic direction and rely on guesswork.
- The final software often misaligns with business expectations, leading to wasted time and money.

This book lifts the lid on that black box by presenting a structured, optimized framework for managing your SDLC with clarity, efficiency, and control. You’ll learn how to go from “**raw ingredients**” (*business ideas*) to “**finished dishes**” (*working software*) in a **predictable**, **scalable**, and **strategically aligned** manner. Along the way, we’ll define all the crucial concepts—like **technical debt** and **CI/CD**—so that you can follow the entire development process, even if you don’t have a deep technical background.

***Note:** The goal of understanding the SDLC is not to become technical, but to gain the necessary visibility to make informed decisions on where time and money are being invested, align the team, and ensure that each development effort delivers tangible value.*

In short, our goal is to help you:

1. Recognize the importance of having a “well-configured kitchen” (*the SDLC*),
2. Measure the performance of that “machine” using indicators like time to market, software quality, and ROI,
3. Understand the key terms and best practices so you can lead with confidence,
4. Eliminate the black box effect by shedding light on every step from idea to product.

Welcome to a new level of clarity in software development!

Stories

Theory is only part of the equation. To truly understand the impact of a well-structured Software Development Lifecycle (SDLC), we need to look at real scenarios—projects that succeeded or failed based on how (*or if*) they applied these principles.

In this section, you'll find six short stories based on real companies (*names have been changed*). Each one highlights a different challenge: from building features blindly, to scaling without strategy, or mismatching teams with project types. These aren't just cautionary tales—they're lenses that expose how missing or misapplying SDLC elements leads to unnecessary cost, confusion, and technical debt.

As you read, pay attention to the recurring patterns. You'll see how small decisions early in the lifecycle—especially in the Product Discovery phase—can shape the success or failure of an entire system. More importantly, you'll see how having a clear framework could have changed everything.

Story 1: When Development Defines the Product

BetaWave was a small but ambitious startup with only three team members: a CEO, a salesperson, and a developer. They had no formal Product Discovery process. The salesperson, acting as an informal Product Manager, simply relayed feature requests; the developer built them from scratch, never pausing to check whether existing solutions might suffice.

What Went Wrong

Because there was no validation or research, the team created features already available in common ecommerce solutions. This custom code was expensive to maintain, slow to develop and the product lacked a cohesive roadmap. Over time, technical debt piled up, and the team realized they could have launched faster and cheaper by leveraging existing tools.

Lessons Learned

- **Missed Cost Savings:** By not validating solutions up front, they built costly, bespoke features instead of using off-the-shelf plugins or frameworks.
 - **Discovery Failure:** A proper Product Discovery phase would have revealed existing solutions and cut development costs drastically.
 - **Low Financial Risk, High Uncertainty:** They tried to do everything from scratch as though it was a validated system, but they were actually in an MVP phase—yet without the lean approach that MVPs typically requires.
-

Story 2: The Billing Company and Directionless Development

Invoicely was a billing company built on multiple microservices, each communicating via REST APIs (*a standard way for different software parts to talk to each other over the internet, like placing orders between kitchen stations*) and connected to a separate frontend. They had no Product Discovery stage and no dedicated sales or business team to provide market insights. As a result, developers made decisions in a near vacuum, guided mostly by code rather than validated business needs.

What Went Wrong

At one point, the team needed to unify a “loan eligibility” feature—originally scattered across various microservices—into a single service. This unification process essentially became a late and expensive form of Product Discovery, because nobody had documented or validated how the eligibility logic should really work. Each microservice had been developed by different people at different times, with no central vision.

Developers spent weeks analyzing code to infer the eligibility rules, since there was no business or product expert who could confirm the real requirements. As a result, the actual logic ended up being “dictated by the code,” rather than by informed business knowledge. Many iterations followed, each attempting to consolidate or rewrite parts of the service, yet no short-term success was achieved. By the time the real stakeholders got involved, the system was already overcomplicated, costly to maintain, and misaligned with the core business model.

Lessons Learned

- **Unstructured Discovery at the End Is Costly:** Trying to figure out the product vision via existing code is an expensive, high-risk approach. If you don't define requirements early, the code itself becomes the de facto “source of truth,” often filled with assumptions and hacks.
 - **Misplaced Ownership:** With no clear product or business lead, developers took on discovery tasks. They lacked the knowledge or resources to define coherent business logic, causing confusion and rework.
 - **Complexity Begets Complexity:** Scattered microservices, each built by different people, led to a decentralized system that was impossible to unify smoothly. A well-defined Product Discovery stage could have established consistent eligibility rules from the start and reduced both technical debt and wasted effort.
-

Story 3: The Dissonance Between Team Structure and Project Goals

LegalTech specialized in automating legal workflows for malpractice cases. They had separate teams for Product Discovery and Software Delivery. However, they attempted to run a **rapid experimentation initiative** with a team designed for **robust, long-term delivery**—complete with thorough architecture, reliability checks, and multi-layer testing.

What Went Wrong

Because the team was oriented toward stability and durability, they relied on heavier processes and rigorous testing. This approach slowed down the short, iterative cycles that fast experiments typically require. The Product Manager grew frustrated, seeing multiple delays. Ultimately, the mismatch between “**quick experiment**” goals and “**heavy delivery**” practices created unnecessary friction.

Lessons Learned

- **Team Configuration Mistake:** They assigned a stable, more formal process to a project meant for quick experimentation.
 - **Time-to-Market Loss:** Combining a rapid prototype mindset with a heavier architecture/test approach caused delays and increased costs.
 - **Key SDLC Failure:** Recognizing the nature of the project from the start—especially its scope, complexity, and desired speed—was critical. Without this clarity, they couldn’t align the right team configuration or methods.
-

Story 4: Building a System Unknowingly

FinCorp developed a lending platform for large-scale B2B deals. They believed they were building an MVP—minimally viable and flexible—while in reality they needed a system with strict non-functional requirements (*e.g., the ability to install on clients’ on-premises servers*).

What Went Wrong

Months into coding, the Product Manager discovered the platform had to run outside AWS (*a cloud service provider*). Retrofitting an AWS-exclusive design to on-prem servers proved expensive. This mismatch in architecture and actual requirements created major rework and cost overruns.

Lessons Learned

- **Misidentified Project Type:** FinCorp labeled it an MVP but faced enterprise-level constraints typical of systems.
 - **Costly Rework:** Late discovery of on-prem requirements forced an architectural overhaul, incurring high expenses.
 - **Discovery & Delivery Gap:** Technical decisions happened in isolation from actual business constraints, showing the importance of integrated feedback early on.
-

Story 5: The Hybrid That Didn't Work

MediCore provided virtual payment cards for insurance companies. They had separate Product Discovery and Software Delivery teams, but never explicitly defined which projects were “quick experiments” vs. “long-term systems”. Both approaches got mixed in a single codebase.

What Went Wrong

Some parts of the product were coded rapidly like an MVP, others were treated as stable, permanent systems. The resulting architecture was inconsistent, and technical debt skyrocketed. Eventually, the mix-and-match approach ballooned costs and slowed any real progress.

Lessons Learned

- **Hybrid Confusion:** Blending MVPs and systems in one product typically combines the worst of both approaches—incurring technical debt without the advantages of quick validation or long-term stability.
 - **Financial Burn:** Constant refactoring plus infrastructure expansion drained budgets.
 - **Framework Failure:** If they had designated separate streams—one purely for MVP experiments, one for robust system building—they could have avoided these costly collisions.
-

Story 6: The Dual-Track Success

NovaCredit had a novel idea for a financial service but also recognized the need for a stable architecture once the idea was validated. From Day One, they planned two tracks:

- **Labs:** A fast, lightweight MVP to gather user feedback and secure a pilot customer.
- **Product:** A robust back-end system built in parallel, integrating lessons learned from the MVP as they emerged.

What Went Right

Once NovaCredit secured its first pilot client, the MVP served as the perfect sandbox to validate requirements, gather real-time feedback, and pivot quickly. Meanwhile, a separate team—composed of infrastructure specialists, QA, and architects—worked on the long-term system. Both teams maintained open communication: every feature tested and validated in the MVP that proved valuable was reimaged using the insights gathered and re-implemented with best practices in the scalable system.

Lessons Learned

- **Lower Financial Risk, Lower Rework:** The MVP was inexpensive and handled uncertainties, while the more expensive system work happened only after initial validation. This dual-track approach minimized total rework.
- **Clear Cost-Benefit:** The client accepted potential MVP hiccups (*late-night bug fixes, incomplete features*) under a pilot contract, in return for shaping a final product that would meet enterprise standards.
- **Smooth Transition:** By the time the MVP's user flows were confirmed, the robust system was almost ready, ensuring a fast switchover. This approach drastically reduced technical debt and boosted ROI.

Final Note on Project Risk

- **MVP:** Generally lower financial risk because you're investing minimally upfront, but higher uncertainty about the final product-market fit.
 - **Scalable System:** More significant up-front investment and effort, but lower product risk once validated—smoother to operate, scale, and maintain.
-

These six stories demonstrate how critical it is to identify your project type, set up the right teams, and maintain a tight feedback loop between Discovery and Delivery. By learning from both failures and successes, CEOs and founders can optimize ROI, reduce technical debt, and avoid expensive misalignments.

Key Takeaways

1. **Product Discovery is essential:** Each story highlights how skipping proper discovery leads to poorly validated features and wasted effort.
2. **Match the team and the project type:** Using the wrong team structure for the project's goals causes delays and frustration.
3. **Define the project type early:** Misalignment between an exploratory initiative and a system-oriented team leads to confusion, delays, and poor execution.
4. **Adopt a clear framework (SDLC):** A well-defined process for discovery, design, and delivery helps prevent pitfalls in both technical and business aspects.
5. **Ensure bidirectional clarity between PMs and Leads:** Failures often stem from unclear requests or unvalidated assumptions at the intersection of Product Discovery and Software Delivery. Explicit communication is key!

By aligning each project with the **right methodology, team structure, and discovery strategy**, companies can avoid the mistakes highlighted in these stories.

The following framework provides a clear path to make those decisions consciously — from project classification to delivery practices.

Chapter 1: The Software Development Machine

1.1 The SDLC as a System of Nested Components

The Software Development Lifecycle (SDLC) can be understood as an industrial machine, where the input is a business idea, the process is the development team, and the output is a finished product. This analogy helps CEOs and founders see software development as a set of interconnected gears that must work together smoothly to deliver real value.

In practical terms, the development team operates like a black box: a business concept comes in, it is processed through various stages, and a final outcome is delivered. However, not all business ideas need the same setup. Different types of projects require distinct processes, team configurations, and investments to ensure the best results. As we saw in

Story 1 (*When Development Defines the Product*), neglecting a tailored configuration can lead to wasted resources and misaligned outputs.

1.2 Types of Projects in the SDLC

1.2.1 Type A Projects: Experimental or MVPs

These projects are about fast exploration. Their primary goal is to:

- Test market hypotheses quickly.
- Evaluate a feature's impact.
- Obtain rapid validation with minimal initial investment.

Important Clarification on “Low Risk”: In Type A, “low risk” refers to the initial financial outlay (*time, money, and resources*). It does **not** mean guaranteed success. An MVP can still fail if it doesn't address a genuine user need. However, because the budget and time spent are small, the failure won't cripple the entire business.

Example of an e-commerce MVP

Imagine an e-commerce startup wanting to verify if customers will buy a new category of products. They build a basic landing page and connect it to a payment gateway in just two weeks. Minimal code, minimal design—just enough to see if people actually purchase. If the idea doesn't gain traction, the loss is small and the team can pivot quickly.

1.2.2 Type B Projects: Scalable Systems

These projects focus on stable, maintainable software (*meaning it's easier and cheaper to update or fix over the long term*) capable of handling larger user bases or critical operations. They prioritize:

- High code quality (*leading to fewer bugs and easier changes*) and robust architecture (*a solid foundation that supports stability and future growth*).
- Reliability under heavy load.
- Long-term maintainability (*ease of updates and fixes*) and ongoing updates.

Example of an e-commerce system for 100k Users

An established retailer planning for 100,000 concurrent customers (*users accessing the system at the same time*) needs more than a simple MVP. The system must handle spikes in traffic, integrate with inventory databases, and ensure secure transactions. Failure here

could damage brand reputation and revenue—hence the emphasis on scalability (*the ability to handle growth*) and resilience (*the ability to recover quickly from failures*).

1.3 Configuring the Machine for Each Type of Project

Because Type A and Type B have different aims, the team structures and work processes also diverge:

- **Type A:** Small, flexible teams with minimal bureaucracy and fast decision-making. One developer plus a product manager can often launch a functional MVP. Low initial cost, fast iteration (*repeated cycles of building, testing, and learning to improve quickly*).
- **Type B:** Multidisciplinary teams (*developers, quality experts, infrastructure specialists*) with formal processes to ensure stability. Higher initial cost, but essential for reliability and scaling.

Note: *The higher initial investment in a Type B project (team, processes, architecture) is not a ‘luxury’—it’s a strategic investment in your core business’s future stability, scalability, and maintainability. Cutting corners here usually costs much more in the mid- to long term.*

Diagram/Table: Key Differences (Type A vs. Type B)

Characteristic	Type A Project (MVP)	Type B Project (System)
Main Objective	Rapid validation with small investment	Reliable, long-term software for critical business
Team Setup	Minimal roles, quick coordination	Specialists (QA, DevOps, etc.), structured flow
Time to Market	Often a few weeks to months	Months or longer, with extensive planning
Infrastructure	Lightweight or temporary	Robust, designed for high availability
Risk Focus	Low financial investment	Risk in failing user & performance expectations
Use Case Example	Testing a new feature or market idea	Banking system, large e-commerce

This simple comparison helps you, as a business leader, decide if you need a quick prototype to test the waters or a reliable framework designed to support thousands of users.

Real-world examples make these distinctions clearer. In **Story 4** (*Building a System Unknowingly*), the team believed they were building a quick prototype to validate their idea, but the project actually required enterprise-level robustness from the beginning—including strict non-functional requirements like on-premises deployment, scalability, and compliance. These demands were too significant to justify starting with a lightweight MVP. The mismatch led to costly rework and delays. Had they correctly assessed the project's constraints and requirements upfront, they could have chosen a more suitable approach, saving significant time and resources.

1.4 Working Backwards from the Desired Output

A common mistake is jumping into development without specifying the result you want. Instead, start by defining:

1. The *goal* (e.g., “Verify if new product X has enough demand.”).
2. The *type* of project (A or B).
3. The *team configuration* that aligns with the outcome.

For instance:

- If your aim is just to see if users will buy your new product, set up a lean Type A team, skip heavy processes, and go live quickly.
- If your aim is to run a robust e-commerce platform for thousands of daily orders, invest in a Type B team structure: QA, DevOps, architecture design, etc.

Conclusion: Always Adapt to the Goal

The secret to effective software development is assembling your “machine” in line with the final objective. By understanding whether a project is Type A (*fast validation*) or Type B (*sustainable system*), you reduce waste and ensure your product meets its real market and technical needs.

In the next chapter, we’ll explore how these differences shape your approach to **Product Discovery** and **Software Delivery**—two processes that keep your ideas aligned with true user demands and long-term growth.

Remember:

- Type A’s “low risk” lies in spending less upfront, not in a guaranteed outcome.
 - Use minimal features for rapid tests if speed and feedback are paramount.
 - Invest in robust architecture for mission-critical solutions, where failure is simply too expensive to risk.
 - Always decide on the project type before writing code; it’s far easier to pick the right path up front than to rebuild later.
-

Chapter 2: Types of Projects

The Software Development Lifecycle (SDLC) is not a one-size-fits-all process. Depending on the final goal, the type of software to be built, and environmental conditions, projects can be classified into two broad categories: **Type A Projects (Experimental / MVPs)** and **Type B Projects (Systems / Scalable)**. Each category requires a different approach to team organization, development processes, and quality standards. Understanding these differences is key to avoiding unnecessary friction and ensuring that resources are allocated efficiently.

2.1 Type A Projects: Experimental / MVPs

These projects take an **exploratory** approach, where the main objective is not stability or scalability but **fast validation** of an idea with minimal effort. They are characterized by:

- Short development time.
- Rapid iterations and immediate feedback cycles.
- Low cost in infrastructure and maintenance (*the underlying servers, databases, and services needed to run the software*).
- Maximum flexibility in requirements.

Examples of Type A Projects

- **Market tests:** Checking if a product has demand before investing in a full-scale solution.
- **Feature validation:** Launching a simplified version of a new feature to gauge adoption.
- **Social impact pilots:** Pilot programs in sectors where technical or social viability is still unclear.

Two Clear Examples of Type A in Action

Example 1: Type A MVP Within an Existing System (Type B)

Imagine you already have a running e-commerce platform with stable, mission-critical features. You want to **test a new loyalty program** without risking the stability of the main system or investing heavily in infrastructure right away.

1. **Minimal Front-End:** Build a simple, separate page or pop-up within the existing platform that displays loyalty points for a small subset of users.
2. **Manual / Serverless Back-End¹:** Instead of complex APIs (*Application Programming Interfaces, it is a standard way for different software components to communicate with each other*), track user points in a spreadsheet or with a single serverless function (*code that runs on demand without needing dedicated servers, often simpler and cheaper for basic tasks*) updating data for this pilot group.

¹ A back-end is the hidden part of the software handling data processing and logic.

3. **Limited Scope:** Only show this feature to a small percentage of users (*e.g., internal staff or a beta group*). Quickly gather feedback to see if they engage with the loyalty program.
4. **Fast Iterations:** Based on feedback, adjust the rewards logic or interface without the overhead of large-scale developments.

This approach aligns with **The Lean Startup (Eric Ries)** philosophy: experiment fast, measure actual behavior, and iterate cheaply—verifying the feature’s value before investing in a fully integrated solution.

Example 2: Type A MVP From Scratch (No Existing System)

Now consider a brand-new idea for a job referral platform that does not exist yet. You want to validate demand before committing to a big development effort.

1. **Minimal Front-End:** Create a simple landing page where users can submit referrals or sign up. The goal is to validate demand, not to showcase a complete product.
2. **Serverless or Manual Tasks:** Use a single serverless function to store submissions. For tasks like validating referrals or sending notifications, do them manually or with minimal automation.
3. **Acquire First Users:** Invite a small group—friends, colleagues, or a targeted audience—to try it. Refine the user flow based on their direct feedback.
4. **Iterate or Pivot:** If there’s interest, invest further. If not, you’ve minimized wasted resources and can pivot sooner.

Both examples underscore how **Type A Projects** emphasize **speed, minimal infrastructure, and rapid feedback** over heavy investments. They let you test market or user interest quickly and pivot as needed—core aspects of the MVP mindset. This approach reflects well-known techniques from *The Lean Startup*, widely applied by multinational enterprises to achieve rapid validation and growth. We recommend it for a deeper

understanding of how these proven methods can be extended specifically to software development.

2.2 Type B Projects: Systems / Scalable

Unlike experimental projects, **Type B Projects** are designed to be **solid, maintainable solutions** over the long term. They focus on quality, scalability, and stability, with a more formal development approach. They are characterized by:

- Robust, scalable infrastructure.
- Stricter quality and security standards.
- Architectural design aligned with long-term growth.
- Significant investment in maintenance and support.

Examples of Type B Projects

- **Banking and fintech platforms:** Where security and stability are non-negotiable.
- **Complex enterprise systems:** ERPs, CRMs, or large-scale internal platforms requiring broad integrations.
- **Established software products:** Systems that have gone beyond experimentation and need sustainable design.

Example: A Real-Time Settlement Platform in Finance

Suppose you want to build a **real-time settlement platform** for microtransactions in the financial sector—handling hundreds of transactions every minute for a bank or fintech. Here, stability, auditability (*the ability to reliably track who did what and when within the system*), and compliance requirements are non-negotiable from day one:

1. **Stringent Regulatory Requirements:** Financial systems must meet regulations (*like KYC – Know Your Customer, AML – Anti-Money Laundering, or PCI-DSS for payment card security*). There's no “basic” version you can ship quickly; security, encryption (*scrambling data so it's unreadable without a secret key, protecting sensitive information*), and user authentication must be comprehensive from the outset.
2. **Audit Logs and Traceability²:** Every operation—from microtransactions to large deposits—must have a transparent audit trail. A partial or manual logging process (*often okay for an MVP*) is insufficient; everything must be tracked in real time and be

regulator-ready.

² *Traceability ensures every significant action within the system can be followed and verified*

3. **Scalable, Redundant Infrastructure:** Clients expect 24/7 availability, so you need failover mechanisms (*automatic backup systems that take over if a primary component fails*) and load balancing (*distributing incoming user traffic across multiple servers to prevent overload*) from day one. You can't afford downtime or big refactors typical in MVP-style projects.
4. **Robust Testing and Certification:** Performance, security, and integration tests are essential to comply with legal frameworks. Thorough acceptance criteria (*specific, measurable conditions that must be satisfied for a feature to be considered complete and correct*) must be met prior to launch.

A quick **Type A** approach (*minimal, short-term validation*) would be risky in a financial environment. Instead, **Type B** invests heavily in reliable architecture, formal processes, and strict security—consistent with the guidelines in **Clean Architecture (Robert C. Martin)** and **Domain-Driven Design (Eric Evans)**, which emphasize long-term robustness in mission-critical domains.

2.3 Key Differences Between Type A and Type B Projects

Below is a quick reference table comparing the most important characteristics of Type A (*MVP*) and Type B (*Scalable*) projects. Keep these distinctions in mind—and ideally visible for your team—since each row highlights how differently these two approaches handle objectives, costs, quality, and risk. By reviewing the table whenever you start a new project (*or reevaluate an ongoing one*), you'll have a clearer picture of the trade-offs each option entails and can align your team's efforts more effectively.

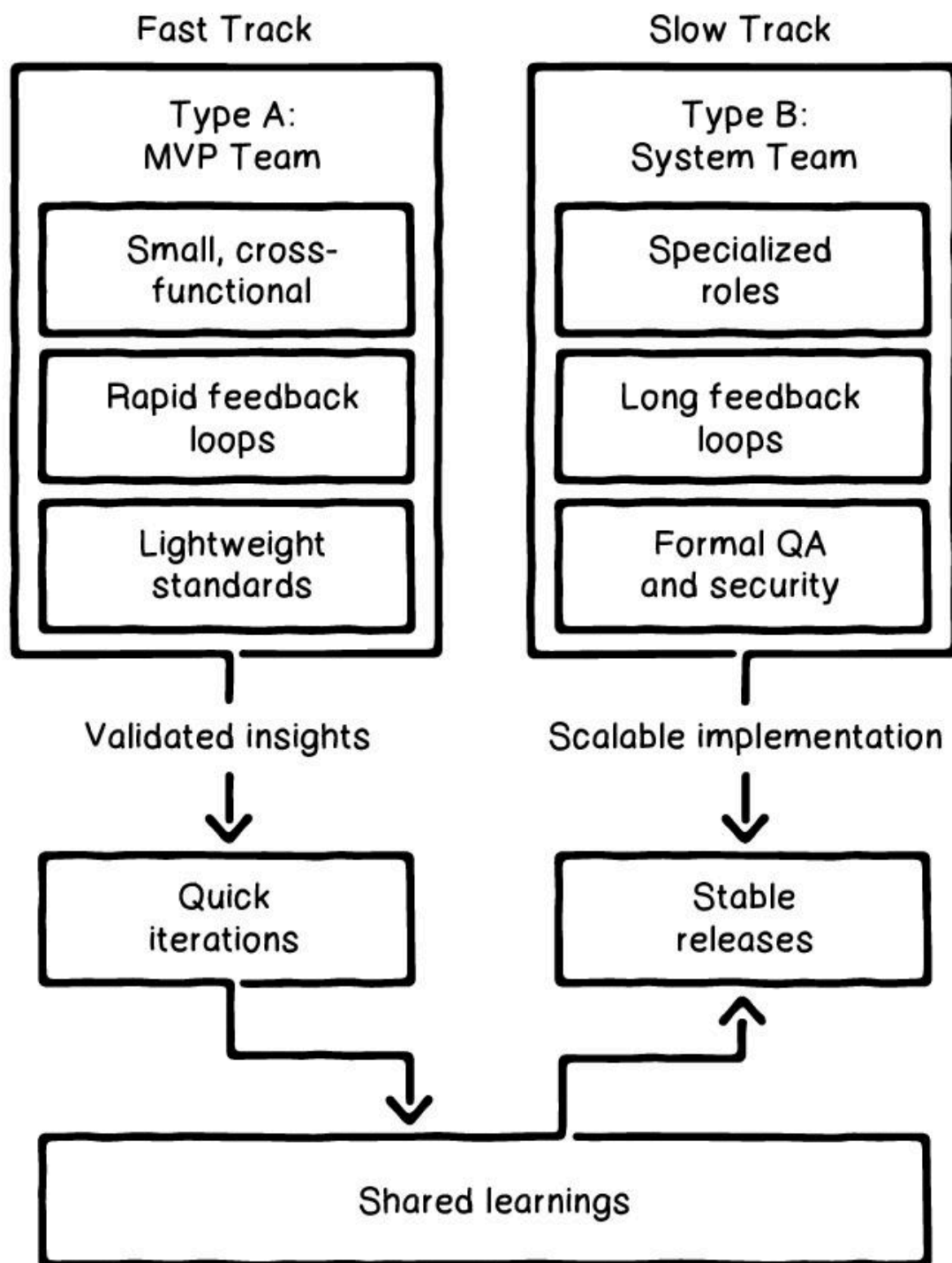
Charasteristic	Type A Project (MVP)	Type B Project (System)
Objective	Fast validation & learning	Stability & scalability
Development Time	Weeks or a few months	Months or years
Code Quality	Flexible, rapid iteration	Optimized and thoroughly tested
Infrastructure	Minimal essentials	Robust and maintainable
Team Management	Small, agile teams	Specialized, possibly larger teams
Initial Costs	Low	Higher, but justified

Dynamics Between Project Types

This diagram highlights the contrast between Type A and Type B teams: MVP teams are small, cross-functional, and built for speed, while system teams are specialized and focused on reliability.

The arrows are intentional. In this book's approach, the recommended practice—if both project types coexist—is to let Type A projects feed Type B systems. That's why Quick Iterations (*from MVPs*) and Stable Releases (*from scalable systems*) converge in Shared Learnings.

These learnings help inform architecture, user experience, and technical decisions in the long term. We'll explore later how this loop can be formalized, but for now, keep in mind that Type A should never be treated as a shortcut to production, but rather as a learning pipeline that enhances Type B over time.



Focus Comparison

This comparison illustrates how smaller, rapid-iteration teams (*often cross-functional, meaning members possess diverse skills needed to complete work without excessive handoffs*) differ from larger, specialized teams that focus on high reliability and formal processes.

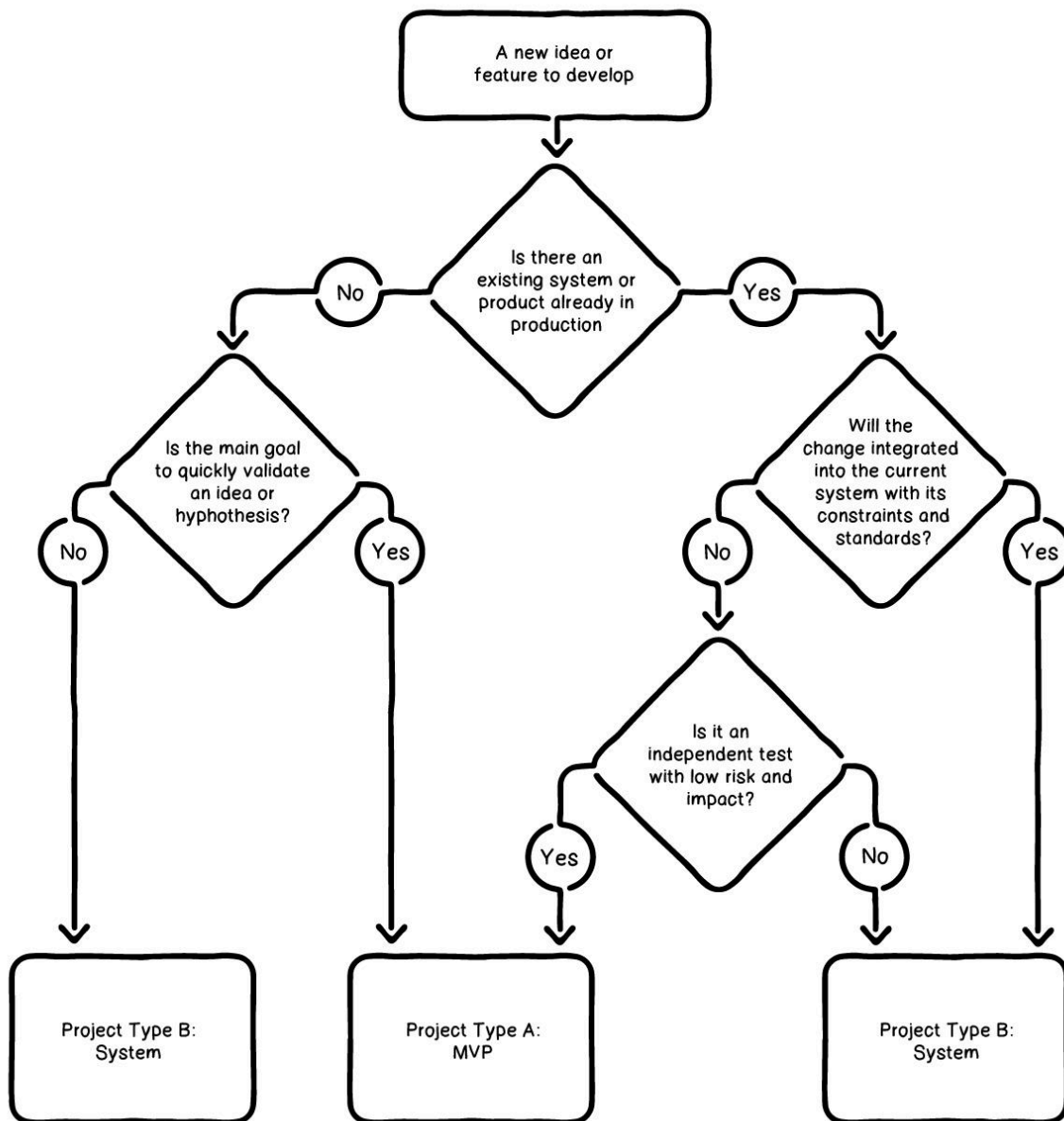
2.4 How to Choose the Right Approach

Before starting, define **what** type of software you're building. A common mistake is to aim for a stable system from day one when the idea is still unvalidated, or conversely, to keep an MVP alive for too long, running it as if it were a final product. Another frequent pitfall is adding new efforts to an existing team without clarifying if the outcome needs a quick experiment (*Type A*) or a robust product (*Type B*).

Ask these questions:

- **Is your main goal to validate a hypothesis quickly?** Type A is a perfect fit.
- **Will the software be the operational backbone of the business?** Then Type B is essential.
- **How big is your budget—and how much risk can you tolerate?** Type A tolerates failures and pivots; Type B requires more upfront investment and careful planning.

Decision Flow – Type A vs. Type B



Note on Risk and Impact:

In this flowchart, risk refers specifically to **technical or architectural risk**—that is, what could happen if the functionality breaks or causes regressions. It does **not** refer to business risk or opportunity cost.

Impact, on the other hand, refers to the **number of users or systems affected**. A low-impact experiment might be limited to 10 internal users, while a high-impact change would affect thousands of customers or the

core business flow. These definitions help determine whether an idea can be tested as a Type A Project or must be treated as a full Type B System.

Place this flow where stakeholders must decide between a quick, exploratory build (*Type A*) or a long-term, mission-critical system (*Type B*).

Remember

If you're consistently forced to "pivot" or "rebuild" because you picked the wrong approach early, the cost grows exponentially. If you **start** with an MVP mindset for something that truly needs robust, long-term solutions, you'll pay for large-scale refactoring later. If you build a Type B system up front for an idea that isn't validated, you may lose opportunities by moving too slowly.

2.5 Why a Hybrid Approach Is a Bad Idea

A recurring error is trying to **mix Type A (Experimental/MVP) and Type B (Scalable System)** inside the same project—believing you can get the "best of both worlds." In practice, it often yields the worst of both:

Note: *Warning! Avoiding the 'hybrid trap' is crucial. Trying to be fast and cheap (Type A) and robust and scalable (Type B) at the same time and in the same codebase almost always results in an unstable, expensive, and slow-to-evolve product. Force yourself to choose a clear approach for each initiative.*

- You lose the **fast time-to-market** benefits of Type A because the process isn't lean or iterative enough.
- You compromise the **stability** of Type B because the system isn't truly designed for long-term maintainability.
- Teams get stuck with contradictory goals: be quick and cheap, yet stable and future-proof. Neither is done well.

Key Point: From this book's perspective, it's best to **avoid** a single hybrid project. If you do both in one codebase—without boundaries—costs and frustrations pile up. Decide early: either adopt the fast-lane approach (*Type A*) or invest in the robust path (*Type B*).

What If an MVP Must Become a System Later?

- In many famous startup stories—such as **Airbnb**, which began as a simple landing page renting out an apartment—what started as an MVP ultimately transitioned into a

fully scalable system. That transition can be done, but it **usually involves paying off a hefty amount of technical debt** or rebuilding large parts of the code.

***Note:** If an MVP (Type A) is successful and needs to scale, that's great news! But be aware: transitioning to a robust system (Type B) will require significant investment (time and money) for refactoring or rebuilding. Plan for this 'second cost' if you anticipate growth from the beginning.*

- This book's approach: if you know from the start that success will require a robust solution, it's often wiser (*and cheaper in the long run*) to **begin with a dual-lane mindset—Type A** to validate quickly in parallel, and **Type B** ramp-up once you have enough confidence. Doing so prevents a massive “tech debt invoice” later.
- However, if you skip all Type B practices at the start (*or vice versa*), you will inevitably pay a high price—either in **refactoring** if the MVP succeeds or in **missed opportunities** if you over-engineer something that nobody ends up using.

Real-World Note: Airbnb famously started as a bare-bones site with most tasks done manually. While it's a success story, they later had to invest heavily in re-architecting for scale—essentially paying the technical debt of that MVP.

This book's stance is that, while transitioning from Type A to Type B is possible, it's often costlier than planning a more structured approach early on. That doesn't mean you must predict the future—but you **can and should structure your teams and project setup based on the type of project and its context**, maximizing the benefits of both approaches.

Knowing that a transition will likely happen (*if the MVP succeeds*) is critical. The problem isn't the transition itself—it's when teams act on instinct or momentum (“*vibe-driven development*”) and face a massive refactor later as if it were some unexpected twist. **If no effort was made to anticipate or reduce that transition cost, then the “surprise” is only the result of not thinking ahead.**

Conclusion: Choose Your Project Type Early—And Stick to It

Success in software development hinges on choosing (*and **sticking to***) the right approach early. **Type A** focuses on quick validation with minimal resources. **Type B** prioritizes a solid foundation for growth and longevity. Trying to blend these approaches in the same project usually triggers more problems than benefits.

If you **truly** need both fast experimentation and stable architecture, the recommendation is to run them in parallel or in distinct stages—**not** to fuse them. By keeping clear boundaries, you reap the benefits of rapid learning (*Type A*) and reliable scaling (*Type B*) **without** getting trapped in the costly hybrid middle.

In the next chapter, we'll explore how these project types fit into the overall SDLC and how **Product Discovery** and **Software Delivery** can interact to maximize effectiveness and ROI for your business.

Chapter 3: Discovery Meets Delivery

Software development isn't a linear process where you simply define a product and build it. Rather, it comprises two key phases that work together to ensure you build something both **valuable** and **viable**: **Product Discovery** and **Software Delivery**.

3.1 Key Stages of the SDLC

The SDLC is split into two broad stages, each with a distinct but complementary focus:

1. **Product Discovery**: Where you confirm that the product idea solves a real user problem.
2. **Software Delivery**: Where you implement and ship the actual solution.

By treating these stages as **continuous and iterative**—instead of purely sequential—you avoid building software that solves the wrong problem or provides little user value. This mindset aligns with building the **right thing first**, then building the **thing right**.

A **crucial aspect** of Product Discovery is ensuring the problem is defined clearly. Tools such as **Impact Mapping**, **User Story Mapping**, and **OOPSI** (*frameworks and techniques for visualizing goals, understanding user needs, and detailing requirements collaboratively*) help break down the **why** and the **what** before committing to the **how**. This prevents the team from building ambiguous solutions that either miss the real problem or inflate technical costs.

3.2 Product Discovery: Exploration, Validation, and Adjustment

In **Product Discovery**, you determine **what** to build and **why** it matters. You focus on user needs, market opportunities, and real-world problems to reduce uncertainty **before** coding begins.

Note: *Investing time in Discovery (validating the what and why with real users) before developers start coding is not a delay—it's the most effective and cheapest way to avoid wasting valuable resources on features no one will use or that don't solve the right problem.*

- **Define the problem:** Make sure a genuine need exists. Techniques like the “5 Whys” or **OOPSI** (*Outcome, Output, Process, Scenarios, Inputs*) can uncover the actual pain point behind a surface request.
- **Validate hypotheses:** Interviews, prototypes, or short experiments test assumptions at low cost. **Impact Mapping** or **User Story Mapping** helps ensure each feature ties directly back to user outcomes.
- **Iterate based on feedback:** Rapid learning cycles let you refine or pivot without sinking months of effort into the wrong solution.

By the end of this phase, you have a clear understanding of the **real** user problem, initial acceptance criteria, and core scenarios—all of which guide the **Software Delivery** phase.

Example: Discovering the Real Need for a Subscription Feature

Imagine your company wants to introduce a **subscription-based offering**. You suspect it will increase recurring revenue, but you’re not entirely sure what users really want.

1. Ask “Why?” Multiple Times (5 Whys)

- **Why** introduce subscriptions? Because we need consistent revenue and higher customer loyalty.
- **Why** do users want it? Because frequent buyers find it tedious to reorder; a subscription might save them time.
- **Why** is that important? Because we see a high churn rate in repeat orders—they often abandon carts when buying again.
By repeating “why,” you discover that **convenience** and **loyalty** might be your real drivers.

2. Map the Impact (*Impact Mapping*)

- **Goal:** Increase repeat purchases by 20% within the next quarter.
- **Actors:** Existing users who buy repeatedly, plus new sign-ups interested in an ongoing deal.
- **Impacts:** Encourage users to set up recurring purchases by simplifying the process (*one-click subscription*).
- **Deliverables:** A subscription flow in the checkout, automated reminders, an email campaign explaining how to pause or cancel.

3. Outline the User Journey (*User Story Mapping*)

- Visualize the steps: “Browse Products” → “Select Subscription Option” → “Set Delivery Frequency” → “Confirm Payment.”
- Identify the **minimum** steps to validate whether people actually sign up. That “minimum path” could become your MVP for the first iteration.

4. Define the Outcome, Output, and Process (*OOPSI*)

- **Outcome:** Users effortlessly schedule recurring orders, leading to better retention.
- **Output:** A “Subscription Checkout” screen and backend logic that automatically charges and dispatches orders at intervals.
- **Process:** From the moment a user clicks “Subscribe,” the system stores their payment info, sets the next billing date, and sends a confirmation.
- **Scenarios:** New subscription, pause subscription, resume subscription, failed payment.
- **Inputs:** User’s payment details, product quantity/frequency, any promotional codes.

By combining these simple steps:

- You clarify **why** subscriptions matter (*5 Whys*).
- You create a **goal-driven plan** (*Impact Mapping*).
- You **visualize** the user’s path (*User Story Mapping*).
- You **structure** the outcome and concrete deliverables (*OOPSI*).

No deep technical detail is required—just enough **to ensure** that when you do move forward with coding, you’re tackling a **real** user need. This saves costs (*by not overbuilding features nobody uses*) and gives everyone—technical and non-technical—a clear understanding of **what** you’re creating and **why** it matters.

3.3 Software Delivery: Implementation, Scalability, and Optimization

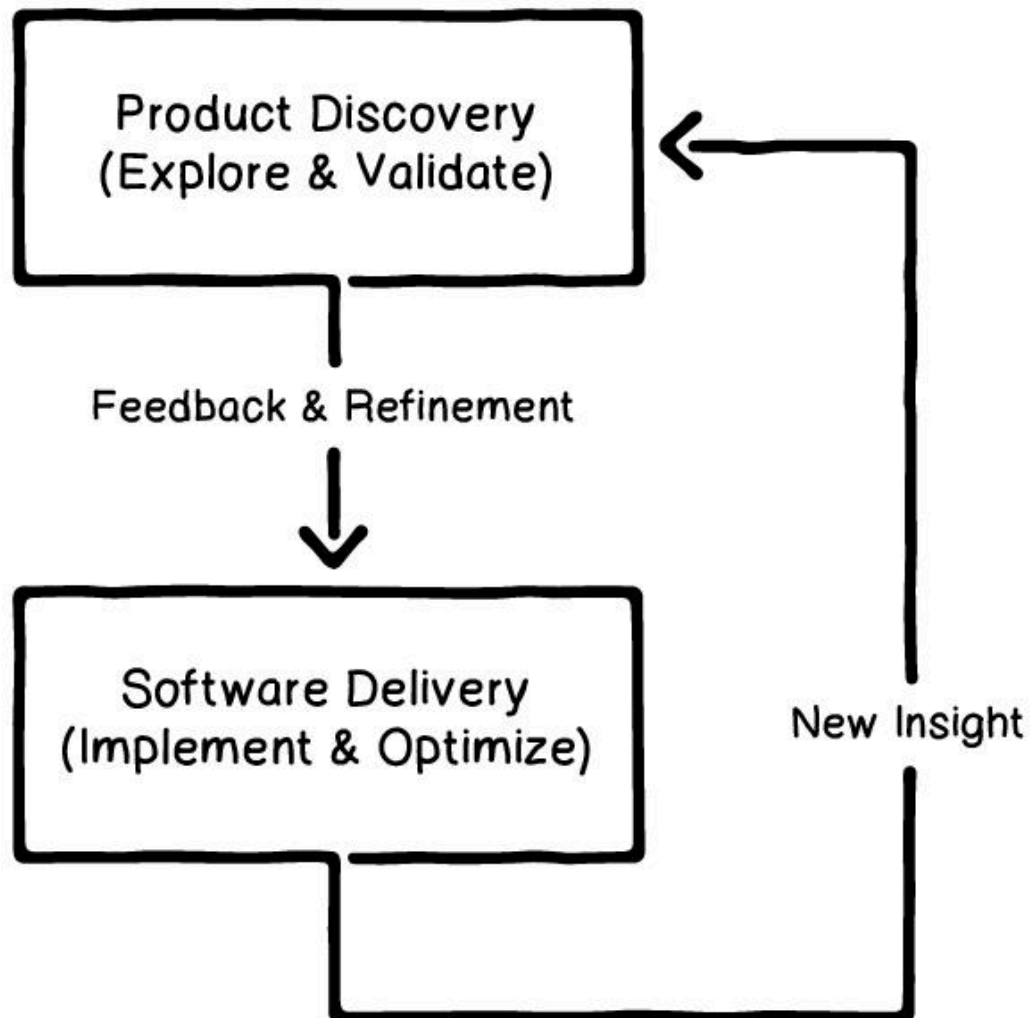
Once you've validated the idea, your team moves on to **Software Delivery**, building the solution in a structured, iterative way. This phase takes different shapes depending on whether you're constructing a **Type A** MVP (*fast, experimental*) or a **Type B** scalable system (*robust, long-lived*).

Key elements include:

- **Clear technical requirements:** Align the team on functional (*what the software should do**) and non-functional (*how well it should do it – e.g., speed, security, reliability*) needs.
- **Iterative development with quality:** Keep velocity (*the team's measured pace of completing work*) high without neglecting code clarity or maintainability.
- **Automation of testing and deployment:** In line with *Continuous Delivery*, automating everything from unit tests (*automated checks on small, individual pieces of code*) to production deployments reduces risk and shortens feedback loops.
- **Monitoring and continuous improvement:** Even early MVPs benefit from observing real usage; feedback data drives iterative improvements.

For **Type A** (*MVP-style*) projects, the priority is speed and validation. For **Type B** (*scalable systems*), you focus on reliability and performance. *Accelerate* (Nicole Forsgren, Jez Humble, Gene Kim) shows that high-performing teams excel by blending both speed and stability without sacrificing one for the other.

Synergy Between Product Discovery and Software Delivery



- **Product Discovery** refines understanding of the real problem using concrete examples and user input.
- **Software Delivery** creates working software, gathers data in production, and loops back fresh insights.

3.4 How Iteration Between These Two Phases Refines Development

One common mistake is treating Product Discovery and Software Delivery as if they're completely separate. In reality, they should overlap and interact:

1. **Validate a concept** with an MVP (*Product Discovery*).
2. **Build an initial release** mindful of early user feedback (*Software Delivery*).
3. **Gather real feedback** from production.
4. **Refine, pivot, or enhance** as needed (*return to Product Discovery*).
5. **Scale or optimize** architecture for ongoing reliability (*advanced Software Delivery*).

Domain-Driven Design (Eric Evans) highlights the value of iterative refinement, especially for complex domains. Early versions focus on fundamental domain concepts, while subsequent cycles evolve the model to meet wider or more complex requirements.

3.4.1 Beyond Sequential vs. Parallel

A common misconception is that **Product Discovery** (*the “what and why”*) happens in strict isolation from **Software Delivery** (*the “how and when”*). In reality, both are tightly intertwined—especially given that a **Type A project** (*MVP*) still requires code to be written, and that code itself generates learning that feeds back into Discovery. Similarly, a **Type B** (*scalable system*) often still needs some form of experimentation or incremental learning as new features are introduced.

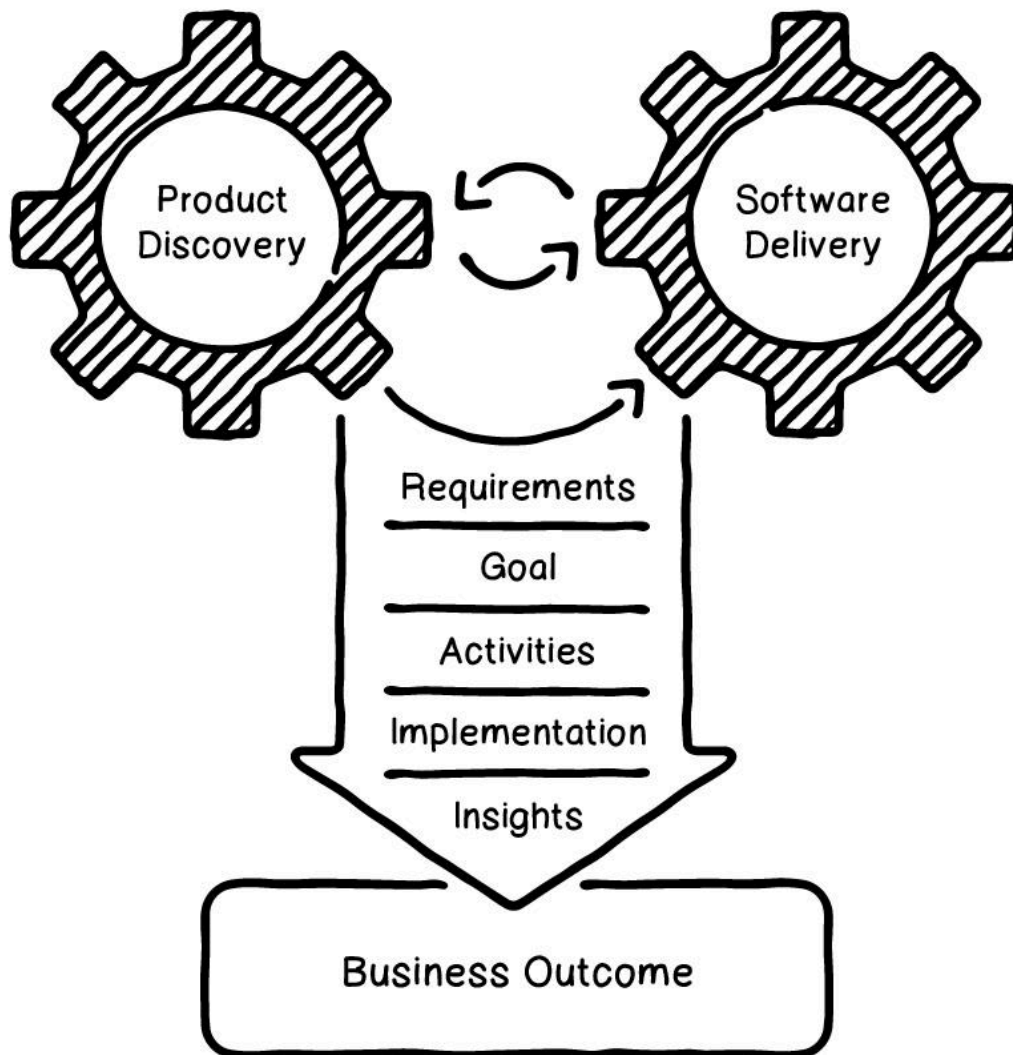
It's neither purely sequential nor purely parallel: it's an iterative, collaborative loop. The difference lies in **what output** we expect from each iteration.

- In **Type A** (*MVP*) mode, the **output** is something we build primarily to learn (*Does this solve the user's problem? Is there market demand?*). Yes, we produce working software, but the *priority* is insight.
- In **Type B** (*scalable*) mode, the **output** is something robust we build to serve actual user demand reliably at scale. We're still learning from user feedback, but the software has a higher bar of reliability and non-functional requirements.

3.4.2 One Continuous Loop, Two Shifts in Mindset

1. **During Product Discovery:** The team (*including developers*) clarifies problems, experiments with prototypes, or builds minimal features. Even in an MVP, that “delivery” portion is present (*someone is coding or configuring tools*). But the *purpose* is primarily to test assumptions and gather evidence about user needs.

2. **During Software Delivery:** The team implements a stable, production-ready version or a new feature. The *purpose* here may be to serve customers at scale (*if we're in Type B*) or to get a refined learning signal (*if still in Type A*).
3. **Both feed each other:** The results from Delivery—whether usage metrics, performance logs, or user interviews—flow straight back to Discovery, influencing the next set of requirements or hypotheses.



Look at the diagram above, which emphasizes how Product Discovery and Software Delivery work together as collaborative gears. Their interaction powers the flow from **Requirements** and **Goals**, through **Implementation**, to **Insights**—culminating in a **Business Outcome**.

What this diagram shows is not a sequence of steps nor two parallel tracks. It's a **collaborative process** where Discovery and Delivery work together toward a shared objective. That objective, however, depends on the **type of project**.

- In a **Type A project**, the goal is to **learn**. Delivery exists, but is lightweight and focused on enabling Discovery through quick experiments or MVPs. The outcome is a functional prototype or partial implementation that generates insights.
- In a **Type B project**, the goal is to **serve**. Discovery still plays a role, but the focus shifts toward building something stable and scalable. Delivery is more formal, emphasizing architectural soundness, maintainability, and production-readiness.

The **type of project must be decided based on the business objective**. Whether you want to validate an idea or deliver a solution at scale determines how these gears are configured, how the team collaborates, and what output is expected from each cycle.

Although the inner mechanics—discovery, delivery, feedback—are always present, it's the **goal** that shifts. That goal aligns both gears in the right mode of operation and ensures they produce the outcome the business truly needs.

Important: This doesn't mean all tasks run in parallel all the time. Certain steps do happen in sequence (*e.g., we define the problem first, then build a solution*). But we avoid rigid "phase gates" because new insights can emerge at any moment. Instead, the team constantly checks: "Are we still learning, or are we implementing at scale? Does this deliver the outcome we need for this iteration?" If not, we loop Discovery and Delivery again, each time with more clarity and a refined goal.

In short, **Product Discovery** and **Software Delivery** are like two gears turning each other in a cyclical process. They happen concurrently enough to stay aligned, yet each iteration may emphasize one or the other more, depending on whether you're focusing on "learn fast" or "serve at scale." The essence is collaborative: everyone (*product, developers, design, QA, etc.*) shares the same overarching goal, and they pivot between learning-driven Delivery (*Type A style*) or service-driven Delivery (*Type B style*), adjusting the output and mindset accordingly.

3.5 Managing Type A and Type B Projects—Transition Considerations

Many teams begin with a **Type A** project (*an MVP*) to validate a hypothesis. If it succeeds, they realize it needs more than quick prototypes and must be refactored into a **Type B** system. **Refactoring** is inevitable at this point—there's no other sustainable way to handle the load or fulfill strict non-functional requirements. As *Working Effectively with Legacy Code* (Michael Feathers) and *Refactoring* (Martin Fowler) explain, it's far less risky to improve code incrementally than to rebuild everything from scratch.

Note: *If your product started as an MVP (Type A) and now needs to handle more users or meet strict requirements, refactoring (improving internal code*

structure) is not optional—it's necessary for survival and growth. It's time to invest in turning it into a sustainable Type B system.

If You Didn't Use the Framework from the Start

You might only discover you need a full-fledged **Type B** approach after the MVP is already in production. For instance, a **finance startup** could begin with a **serverless** or “spaghetti” MVP, only to discover a year later that clients demand a more robust, **containerized** system (*where software is packaged in standardized, self-contained units called containers for easier deployment and scaling*) with CQRS (Command Query Responsibility Segregation) to handle higher transaction volumes and on-premises deployments (*installing the software directly on the client's own servers, rather than using the cloud*).

This is exactly what happened in **Story 4**, where the team had to evolve their initial prototype into a scalable production system once it started gaining real traction.

That transition demands:

- **Structured refactoring** (*cleaning up the original MVP's code so it can scale*).
- **Architecture upgrades** (*e.g., migrating from simple serverless functions to container-based modules –breaking the system into distinct parts, each in its own container– or a monolithic container –packaging the entire application into a single container– with well-defined boundaries*).
- **Expanded DevOps** (*introducing more formal CI/CD pipelines, monitoring, and security checks*).

Yes, it's “common sense” that you must scale properly if you outgrow an MVP, but teams often underestimate the technical debt. Even if you started “wrong,” paying down that debt is the only viable solution. **Starting over completely** is rarely wise because it's extremely hard to estimate timelines, and you'd lose all the domain knowledge embedded in the existing system.

If You Did Adopt the Framework from Day One

If you started with a **clear separation** of Type A (*for discovery*) and Type B (*for robust, long-lived systems*), you generally won't face a major “transformational” jump. Instead, you maintain **two distinct streams**:

1. **Type A**: Rapid feedback, minimal overhead—just enough quality to validate ideas.
2. **Type B**: Solid engineering for features proven to be worth scaling.

Note: *As discussed in Chapter 2 (Section 2.5), mixing both approaches in a single project usually leads to contradictory priorities. If you need a*

refresher on why hybridizing is risky, revisit that section for more detail.

Avoid integrating Type A's experimental code into Type B's production environment too soon. This prevents a large-scale rewrite later because you never have to force a hasty MVP to become a mission-critical system overnight.

Conclusion: Two Gears, One Machine

Balancing **Product Discovery** (*finding the right thing to build*) and **Software Delivery** (*building it correctly*) is fundamental for effective software development. By seeing them as **interdependent**, you ensure each feature is grounded in a real user problem, validated with minimal risk, and built to last as needed.

- **Starting fresh with this framework:**

Let **Type A** (*MVP*) experiments feed proven ideas into **Type B** (*robust systems*). You won't waste time forcibly transforming a quick prototype into a massive, stable product.

- **Adopting it midstream:**

If you already have a successful MVP, expect to **refactor**. Migrating to a Type B system inevitably means paying down technical debt, strengthening architecture, and adjusting team processes. It's far better than a risky rewrite from scratch.

References like *Accelerate* (Nicole Forsgren, Jez Humble, Gene Kim) emphasize how top-performing organizations handle these two “gears” (*Discovery and Delivery*) seamlessly. Whether you start with a small MVP or aim straight for a scalable product, clarifying **project types** and properly orchestrating these phases ensures you build solutions that truly meet user needs and can evolve without crippling debt or confusion.

Chapter 4: The Team Orchestration and Structure

Success in software development not only depends on processes and methodologies but also on how teams are organized. The ability to correctly align roles within the SDLC can mean the difference between a smooth development experience and one filled with friction and delays. Just as important is clarifying the difference between **Product Manager** and **Project Manager**—particularly in larger projects—so that responsibility for the product's vision and the overall execution of software development don't get mixed up or lost.

4.1 Key Roles in Product Discovery vs. Software Delivery

When structuring a team, it's helpful to differentiate between the roles essential to **Product Discovery** and those crucial to **Software Delivery**. While there can be overlap—especially in smaller teams—recognizing these groups helps avoid confusion and ensures each phase gets the right expertise.

4.1.1 Product Discovery Roles

- **Product Manager:**

Owns the vision and priorities of the product. Typically has deep domain knowledge—for instance, in the healthcare or finance industry—and focuses on validating ideas, understanding market needs, and defining a roadmap that aligns with business goals. A Product Manager may know exactly what medical professionals or financial operators require to succeed **but not necessarily how to steer a technical development team**.

- **UX / UI Designer:**

Translates product ideas into user-friendly interfaces, prototypes, and flows. Critical for validating assumptions about usability and customer value.

- **Business Stakeholders (sometimes combined with the Product Manager in small teams):**

Provide strategic input on business objectives, budget, and market positioning.

These roles aim to quickly validate a product concept or feature set before heavy development begins. In smaller startups, a single person might cover Product Manager and UX duties. However, as the product grows or when dealing with highly regulated domains (*e.g., healthcare, finance*), clarity between who shapes the product vision (*Product Manager*) and who executes the technical aspects becomes vital.

4.1.2 Software Delivery Roles

- **Tech Lead / Software Architect:**

Makes high-level technical decisions, defines the architecture, and ensures that the design meets performance and scalability requirements.

- **Developers:**

Implement the solution based on well-defined requirements and design specifications.

- **QA Engineers:**

Ensure quality through automated and manual testing, focusing on reliability, performance, and overall stability of the solution.

- **DevOps / Infrastructure:**

Responsible for deployment pipelines (*automated steps to release software safely and efficiently*), production environment stability (*keeping the live system running smoothly for users*), and ensuring continuous delivery (*enabling fast, reliable software updates*) and high availability (*minimizing system downtime*).

The actual structure of Software Delivery teams varies significantly based on whether the project is Type A (*MVP*) or Type B (*Scalable*). For smaller, experimental Type A efforts, the essential role is typically the Developer—often a full-stack developer—who can quickly produce prototypes without needing extensive QA or DevOps support. By contrast, Type B projects demand more specialized roles (*Tech Lead, QA Engineers, DevOps*) to ensure the resulting product is robust, maintainable, and scalable. This distinction helps teams allocate resources properly and avoid over-engineering small experiments or under-engineering mission-critical systems.

4.1.3 Project Manager

Bridges the gap between the Product Manager and the development team. This person ensures day-to-day coordination of tasks, manages timelines and resources, and keeps communication flowing. A Project Manager understands high-level objectives from the Product Manager but also has the technical project-management skills to guide a dev team.

In many real-world scenarios, a Product Manager might know exactly what a medical practitioner or insurance company needs functionally (e.g., *compliance requirements, workflow constraints*) but lack the full technical background for planning sprints (*short, fixed-length work cycles, typically 1-4 weeks, common in the Scrum framework*) or orchestrating delivery. A dedicated Project Manager (*or Product Owner in Scrum context*) can translate the product vision into manageable tasks, handle risk, track progress, and ensure the entire effort moves toward successful deployment.

Note: *In very small teams, one person may cover both the Product and Project Manager roles. But as the project scales (especially in Type B scenarios), having a dedicated Project Manager becomes essential to ensure coordination, timeline alignment, and risk management across the team.*

4.2 Strategic Roles and Their Positioning in Different Projects

Beyond simply separating Product Discovery and Software Delivery, the size and nature of the project also influence how roles are combined or specialized:

- **For Type A Projects (*MVP*)**

- A small, highly agile team that quickly validates ideas.
- Often, the Product Manager and UX/Designer drive discovery, while a small group of Developers handles both design and implementation.
- Formal QA or DevOps might be minimal, with developers covering basic testing and simple deployment processes.
- A Project Manager may not be strictly necessary if the scope is small—one person could fill both the Product and Project management roles.

- **For Type B Projects (*Scalable Systems*)**

- A broader set of specialized roles is essential to ensure long-term quality, performance, and stability.
- Product Discovery might still occur (*especially when adding major new features*), but Software Delivery includes dedicated QA, DevOps, and more formal processes for design and architecture.
- A Project Manager becomes critical: coordinating timelines, ensuring different subteams (*e.g., multiple Product Managers if different product lines are involved*) stay aligned, and resolving dependencies (*situations where one piece of work relies on another being completed first*).

By strategically placing the right roles in each phase, teams can avoid duplicate efforts and confusion. In small MVP teams, a single person may cover multiple responsibilities. In more complex projects, each area may be handled by an entire team of specialists.

4.3 Differences in Team Configuration Based on Project Type

Charasteristic	Type A Project (MVP)	Type B Project (System)
Team Size	Small and multidisciplinary	More specialized and structured
Level of Formality	Low, with high flexibility	High, with established processes
Approach	Rapid experimentation and validation	Stability, scalability, and maintenance
Tools	Low-code, quick prototypes	Well-defined architectures, CI/CD, tests
Need for Project Manager	Optional if the team is tiny (1-4 people)	Highly recommended for larger, complex teams

Type A projects often use tools designed for speed, such as quick prototypes or low-code platforms (*development tools that allow building applications with minimal traditional programming, often faster for simple needs*).

Developers are key in technical design tasks, including architecture design and sometimes creating sequence diagrams (*visualizations showing the step-by-step interaction between different parts of the system for a specific task, which we'll see more of in Chapter 7*).

4.4 How to Structure Teams for Product Discovery and Software Delivery

Team structure must adapt to the type of project being executed. For Product Discovery, the goal is quick validation, so lean, multidisciplinary teams work best. For Software Delivery—especially when building long-term, scalable systems—more specialization and defined roles are necessary.

In **Type A projects (MVPs)**, the team is small, focused, and agile. Roles often overlap, and what's essential is speed and flexibility. A typical team includes:

- A **Product Manager**, who defines the product vision and brings deep knowledge of the domain or industry.
- A **UX Designer**, who maps user flows and creates quick, testable prototypes.
- One or two **Full-Stack Developers**, capable of implementing the MVP quickly and adjusting based on fast feedback.

In **Type B projects (Scalable Systems)**, delivery requires robustness, maintainability, and coordination. The team tends to include:

- A **Project Manager**, responsible for planning, prioritization, and coordinating across roles.
- A **Product Manager**, still in charge of the vision, but now working in tandem with the Project Manager to ensure feasibility and progress.
- One or more **Tech Leads or Architects**, designing the software foundation.
- **Developers** specialized in frontend, backend, and database technologies.
- **QA Engineers**, ensuring reliability through automated and manual testing.
- **DevOps Engineers**, managing infrastructure, deployments, and scalability.

4.4.1 Example: Two Team Configurations

Startup Scenario (Type A – MVP)

- A Product Manager and UX Designer collaborate on defining and designing the initial concept.
- One or two Developers build a prototype or simple version of the product.
- Infrastructure needs are minimal and can be handled by external services or basic setups.
- A Project Manager may not be necessary; lightweight coordination can be handled informally or by the Product Manager.

Enterprise Scenario (Type B – Scalable System)

- A Project Manager oversees planning and risk, translating vision into execution.

- Product Management continues shaping the vision and validating features.
- A dedicated UX team refines flows based on user feedback.
- Developers and Architects are specialized by area of expertise.
- QA and DevOps roles are involved from the start to ensure reliability, security, and compliance.

This contrast reinforces a central idea of this book: **team structure should not be static**. One person may wear multiple hats in a Type A setting, but that doesn't scale. As soon as a project enters Type B territory, roles must be clarified and coordinated through proper team orchestration.

4.5 Participation by Role in Each Phase (Type A or B)

The following table provides a clearer view of who is typically involved in each SDLC phase. A “✓” indicates active participation, while “⚠️” suggests partial or optional involvement depending on the project type and context.

Phase	Product Manager	QA/ UX	Developer	Infrastructure	Project Manager
Define Problem	✓	✓	✓		✓ (Type B)
Specify Use Cases	✓	✓	✓		✓ (Type B)
Design User Flow	⚠️	✓	✓		✓ (Type B)
Sequence Diagram			✓		⚠️
Architecture Design	⚠️		✓	✓	✓
Record Key Decisions	✓		✓		✓
Implementation		✓	✓	✓	
Internal Feedback		✓	✓	✓	✓
Review and Delivery	✓	✓	✓	✓	✓
External Feedback	✓	⚠️	✓		✓ (Type B)

- **Product Manager:** Primarily involved in domain-related discovery activities (*problem definition, use-case specification*), capturing feedback, and ensuring alignment with the overall product vision.
- **QA / UX:** Crucial for testing and user experience validation, more active during user flow design, internal feedback, and final review.
- **Developer:** Key in technical design, implementation, and validating feasibility.

- **Infrastructure:** Essential in later stages to ensure reliable deployment, scalability, security, and compliance with domain-specific regulations if needed.
- **Project Manager:** Coordinates the entire process—especially important in Type B or larger-scale projects. In **very small MVPs (Type A)**, the Product Manager might handle basic project management tasks, **or each developer might briefly coordinate planning and delivery for their own small sub-project**. However, this only works when the scope is minimal and there are no complex dependencies.

When working on **Type B or large projects**, it's not feasible to split the Project Manager role among multiple developers without causing confusion and inefficiency. These projects typically require a **dedicated Project Manager (or at least one clearly identified person)** who orchestrates tasks, manages timelines, and ensures cross-team alignment, distinct from the Product Manager's responsibility for **what** is built and **why**.

4.6 Addressing Dissonance from Story 3 & Story 5

In previous stories, certain teams attempted to run an **MVP approach (Type A)** and a **robust system approach (Type B)** with the same team—leading to friction and confusion. A practical solution is to **maintain two work streams** that can run in parallel:

***Note:** Make sure the team structure and processes match the project goal. Using a heavy team/process setup (Type B) for a quick experiment (Type A) creates slowness and frustration. Doing the opposite (MVP approach for a critical system) creates instability and business risk.*

1. **Discovery/Experimental Track (Type A):**

- Rapid feedback, minimal overhead.
- Can be managed by the same developers, but with a lean mindset.
- Usually no formal QA or DevOps, or just very light versions.

2. **Stable/Long-Term Delivery Track (Type B):**

- More specialized, with formal architecture, QA, security, and compliance.
- May include the same developers at different times, but using a more rigorous process.
- Overseen by a Project Manager to handle more complex planning and risk mitigation.

The **key** is not that different people must sit in different rooms, but that **the same people adapt their approach** depending on whether they're validating an idea (*Type A*) or building production-grade features (*Type B*). The Project Manager helps ensure that priorities across these parallel streams align with the broader product vision. This approach avoids forcing a “heavy process” on quick experiments or letting MVP shortcuts seep into mission-critical codebases.

Conclusion: Structure Drives Flow

Team structure must be intentional and adapted to the type of development being performed. In small organizations or MVP contexts, one individual may handle both the **Product Manager** and **Project Manager** roles. However, as projects scale (*Type B*), separating these responsibilities helps ensure that domain expertise (*Product Manager*) and delivery orchestration (*Project Manager*) don't conflict or become bottlenecks.

By clarifying these roles and encouraging flexible collaboration—where the same developer can shift between experimental tasks and production tasks if needed—teams can avoid friction, maximize efficiency, and deliver software aligned with real market needs.

In the next chapter, we address the importance of effective communication within the software development process and how to avoid common mistakes in requirements specification.

Chapter 5: Coordinating Through Communication

The success of software development relies not only on a solid team orchestration and structure but also on effective communication, as it's the foundation of true coordination. Many frictions in the SDLC (*Software Development Life Cycle*) happen because of poorly defined requirements, misaligned expectations, and lack of clarity in business objectives.

5.1 No Handoffs, Just Collaboration

Rather than viewing Product Discovery and Software Delivery as two separate stages with a “handover,” the process should be a continuous, collaborative cycle from the moment the problem is defined to the point when the product is delivered. By eliminating the idea of handing off requirements, both business and technical teams maintain a shared understanding of what needs to be built and why it matters for the user.

Note: *Actively promote continuous collaboration between business (Product) and tech (Development). The idea of 'handing over requirements' like a baton is a major source of misunderstandings, delays, and building*

the wrong product. Your role is to help break down those silos.

5.2 Not All Discoveries Lead to Software Delivery

One critical point that often goes unmentioned is that **not every item uncovered during Product Discovery ends up in Software Delivery**. Sometimes, the best solution to a user problem isn't a line of code or an automated system but rather a non-technical fix—like adding a new process, changing a policy, or hiring external help.

Example: A Non-Software Solution for User Complaints

Imagine your Product Discovery research reveals that many users leave negative public reviews instead of filing complaints via email. Further investigation shows these users have limited access to email or find it cumbersome. While you might initially think of building a sophisticated support portal or automation, a simpler fix emerges:

1. **Discovery Uncovers the Real Need:**

The true problem isn't "lack of an automated support channel." It's that users want to reach someone quickly and easily, but find email inconvenient.

2. **Proposed Outcome:**

Provide a phone number so users can talk to a support agent directly.

3. **Resulting Solution:**

Instead of building a new piece of software, you set up a dedicated phone line or hire a call center. Users can now complain or ask for help in real time, drastically reducing negative public feedback.

4. **No Software Delivery Needed:**

Although you discovered a major user need, the fix isn't a new app or web feature. Software Delivery remains a tool in your toolkit, but here the best answer was non-technical.

This example highlights that **Product Discovery** can point the team toward **any** effective solution. Software Delivery should be viewed as one tool among many. When Discovery reveals that a business or operational change—like hiring a call center—is more efficient, you can skip new software altogether. **The ultimate goal is to solve user problems and deliver value, not to force a software build just because it's part of the SDLC.**

Key Takeaway: Product Discovery should guide you to the right solution—whether that’s a line of code or a phone line. Keeping an open mind ensures you don’t invest in building software where a simpler, non-technical approach would do.

5.3 How to Avoid Errors in Requirements Specification

Errors in requirements are among the most common reasons projects fail or get delayed. The following practices can help your team minimize mistakes. Below, we’ll ground each recommendation with practical examples and insights for CEOs or founders, showing how user stories, **Domain-Driven Design (DDD – an approach for modeling complex business areas accurately in software)**, **OOPSI**, and lightweight validation cycles can clarify the “what” before building the “how.”

Note for curious founders and CEOs

Throughout this chapter, I mentioned several frameworks and techniques like OOPSI, Impact Mapping, User Story Mapping, BDD, Specification by Example, among others. These are not developer magic tricks—they’re structured ways to translate a business idea into clear conversations that lead to high-quality software.

You don’t need to master them to apply the concepts in this book, but if you’re curious about how they work or how to use them with your team, I’m preparing a visual and simple appendix for non-technical founders that will be included in a future edition.

In the meantime, feel free to reach out if you want a direct explanation or help applying them in your company.

1. Define the “What” Before the “How”

- **Establish Context and Background:** Every task or feature should start by clarifying *why* it matters. Include business context, user needs, and the desired outcome or output (*using OOPSI – a framework ensuring clarity by defining these key aspects: Outcome, Output, Process, Scenarios, and Inputs*) to tie each requirement to a tangible business goal. In more complex or strategic projects, **DDD** (*this approach focused on deep understanding of the business domain*) can help you identify bounded contexts (*clear boundaries between distinct parts of your business model within the software*) and establish a ubiquitous language (*a shared, precise vocabulary used consistently by both business and technical teams*) for the domain so that everyone shares the same definitions.

- **Include User Stories Aligned with Real Needs:** A good user story focuses on the user's intention, not the implementation. For example:
 - **Poor Example:** "As a user, I want to log in with username and password so I can access the app."
 - This frames the solution (*username/password*) rather than the true need (*secure access*).
 - **Better Example:** "As a user, I want to access my account securely so I can protect my private information."
 - This clarifies *why* the user wants security, without locking the team into a specific technical approach. The actual solution might be passwordless login, passkeys, or social login—whatever best aligns with the outcome of secure and convenient access.
- **Document Acceptance Criteria and Relevant Info:** Each requirement should list the criteria for success. For a "secure login" story, acceptance criteria could mention compliance standards (e.g., *2FA – Two-Factor Authentication, requiring a second proof of identity like a code from a phone app for enhanced security*), user flows (*password reset*), or other quality measures. Include sketches, flowcharts, or diagrams of infrastructure where helpful. These visuals often clarify complexities that words alone fail to capture.

2. Short Checklist

- **Context:** Why does the user need this?
- **Business Objective:** What strategic or ROI-driven goal does it fulfill?
- **Acceptance Criteria:** Define success conditions or user flows.
- **Example Scenarios:** Use Specification by Example (*defining requirements through concrete examples*) or BDD (*Behavior-Driven Development*) "Given-When-Then" (*a structured way to describe scenarios using examples*) to clarify behavior.
- **Stakeholder Validation:** Ensure all relevant parties review the story.

3. Use Well-Structured Tasks Combining OOPSI, User Stories, and Diagrams

- **Break down the Outcome** (*"securely access my account"*) into specific **Outputs** (*login screens, API endpoints, encryption schemes*).

- **Add Process** steps like what happens when user credentials are submitted, how the user's session (*their logged-in state and activity period*) is maintained.
- **Define Scenarios** like successful login, wrong password attempt, password reset and gather **Inputs** (e.g., *performance constraints, security compliance*).

Tip: For a user story about secure access, you might include an infrastructure diagram showing how authentication requests flow to a server, plus a sequence diagram of the login process. This ensures both technical and non-technical stakeholders understand the high-level mechanics and see how the solution fulfills the user's real outcome (*privacy, data isolation, unique profile*).

4. Concrete Secure-Login Example with OOPSI

- **Outcome:** The user can access a personal, private account.
- **Output:** A user-authentication flow (e.g., *email + password*) that's straightforward but sufficiently secure, given the data's sensitivity.
- **Process:** Register a new account, verify email, sign in, reset password.
- **Scenarios:**
 - **First Registration:** User provides basic information (e.g., *name, email, birthday*), the system emails a temporary password, and the user confirms ownership by logging in the first time.
 - **Forgot Password:** User provides email, system sends a reset link.
 - **Successful Login:** System grants access to private data and user's profile.
- **Inputs:**
 - For registration, the input might be the name, email, and optional profile data (e.g., *birthday*). No password is entered here if you choose to email the user a temporary one.
 - For first login, the user inputs the temporary password and chooses a new, permanent password.
 - For subsequent logins, the user simply provides email and password.

Tip: “Insights” as an Additional Layer

Sometimes you'll also document “Insights” (e.g., *security*

requirements, compliance constraints, or performance limits) that guide your decisions. For example, if the stored data isn't financial or highly sensitive, you might skip advanced 2FA. If it's more sensitive, consider passkeys or multi-factor authentication for stronger security.

5. Validate Requirements with Examples and Prototypes (*Type A* → *Type B*)

- **Lightweight Validation First (*Type A*):** Sometimes you don't need a fully built system to confirm an idea. You can create a simple mock-up using slides, wireframes, or a no-code front-end to demo the user journey. Show how the user would navigate and where they'd authenticate.
 - **Real Example:** A front-end prototype that simulates 'secure login' flows—Magic Link (*a method where users log in by clicking a unique, temporary link sent to their email*), Passkey, or social login. You gather feedback from real or representative users:
 - Is this simpler than a traditional password approach?
 - Is the security level acceptable?
- **Incorporate Feedback into the Final Build (*Type B*):** Once the approach is validated with a prototype, transition to the main implementation. This step ensures you're building a solution that truly meets user needs. The key is that you don't invest heavily in full development until there's clear evidence you're solving the right problem.

6. Review with Technical Teams and Stakeholders Continuously

- **Avoid Handoffs:** Instead of one massive, handover-style “requirements document,” keep a continuous loop: business context ↔ development feasibility ↔ user feedback.
- **Ongoing Alignment:** Have regular check-ins with technical leads to confirm feasibility, cost, and timelines. For instance, if your “secure login” concept morphs into passkeys, verify if your current infrastructure supports it or if you need additional libraries or frameworks.
- **Frequent Feedback:** Encourage stakeholders and actual users to test prototypes or review scenario examples. This ensures you catch misconceptions early, saving time and money.

Grounding It for You

By integrating **context**, a clearly stated desired outcome, user stories aligned with genuine user needs, acceptance criteria, and relevant diagrams, you create requirements that cut through ambiguity. This approach:

- **Connects Requirements to Business Goals:** You're not just logging people in; you're ensuring secure, frictionless access that protects user data and company reputation.
- **Reduces Waste:** Lightweight prototypes (*Type A*) validate ideas before full-scale development (*Type B*), minimizing expensive rewrites.
- **Keeps Teams Aligned:** Stakeholders, developers, and users share a single source of truth.
- **Prevents Overly Prescriptive Solutions:** By focusing on the true user outcome, your technical approach remains flexible, potentially unlocking more efficient or innovative implementations.

This level of specificity—grounded in practical examples—helps any CEO or founder gain confidence that the development team is spending time on what matters. It also makes it easier to gauge progress because each requirement has a clear success measure and direct user or business impact.

5.4 The Importance of Clarity in Problem Definition

Too often, teams invest time and resources solving a symptom rather than the real issue. This disconnect leads to suboptimal solutions, wasted budgets, and missed opportunities. For CEOs and founders wanting to ensure that teams tackle the **actual** problem, here are five practical strategies—followed by a real-world example showing how these strategies work in practice.

1. **Ask “Why?” Multiple Times** (*5 Whys – a simple technique to find a problem's root cause by repeatedly asking 'why'*)
 - Repeatedly asking “Why is this happening?” helps peel back layers of symptoms and assumptions until you uncover the true cause.
 - **CEO Question:** “Have we traced this problem back to its source, or are we just fixing superficial symptoms?”
2. **Root Cause Analysis & Visual Methods** (*systematic approaches to uncover the fundamental cause behind an issue*)
 - Techniques like **Fishbone (Ishikawa) Diagrams** help map out potential contributing factors (*e.g., People, Processes, Tools*).
 - **CEO Question:** “Which elements—people, processes, technology—are driving this problem, and how do they connect?”

3. **Impact Mapping** *(a strategic planning technique that visually connects project deliverables to business goals and desired changes in user behavior)*

- By linking **business goals** to specific **features** and **user behaviors**, you can validate whether a proposed solution truly addresses a core objective.
- **CEO Question:** “What measurable outcomes are we aiming for, and how will this feature contribute to those outcomes?”

4. **User Feedback & Observations**

- Interview real users or customers to see how they currently perform tasks. Observe them in action; sometimes the user’s initial request is a solution guess rather than the actual need.
- **CEO Question:** “Are we solving the user’s real pain point, or just building what they’re asking for because they think it’s the answer?”

5. **Quantify the Business Impact**

- Attach metrics—time saved, revenue potential, user adoption—to each problem so you can prioritize effectively.
- **CEO Question:** “How will solving this problem move the needle for our business goals, and is it worth the investment?”

Real-World Example: “Just Give Me an Excel”

A common scenario unfolds when a user says, “I need an Excel file with all these numbers.” A CEO or founder trying to understand the true need might explore further:

1. **Ask “Why?” Multiple Times**

- *User:* “I need an Excel with these numbers.”
- *CEO:* “Why do you need the Excel?”
- *User:* “I sum them up to put in a separate report.”
- *CEO:* “What do you do then with the report?”
- This iterative questioning reveals that the user wants a single value populated in a specific field.

2. **Root Cause Analysis**

- At first glance, it seems the user wants an Excel export. The “5 Whys” show the deeper cause: the user is forced to do manual data manipulation because the software doesn’t automate it.
- A quick root cause diagram (*Fishbone or a simple bullet list*) might reveal that the system is missing a “sum & auto-populate” feature.

3. Impact Mapping

- Before coding anything, tie the user’s request back to a business goal. For instance, a key metric might be *time spent on repetitive tasks*. If automating this sum saves the user 20 minutes a day, and that user handles hundreds of similar tasks per month, the overall impact on productivity is significant.

4. User Feedback & Observations

- The CEO sits next to the user (*or does a screen-share*) and watches the user export numbers, open Excel, manually sum them, and paste them in another application. The user is actually “solving” their own problem in a cumbersome way. Seeing this in real time clarifies that the real problem is *manual effort*.
- Instead of giving them a better Excel export, you discover that a more direct solution would be **automatically placing the summed number** into the needed field—no Excel required.

5. Quantify the Business Impact

- Calculate how many hours this manual step wastes per week, multiply by the user’s hourly cost, and see how automation translates to real savings. If the same action is repeated in multiple departments, you can estimate the scaling benefits.
- Now you’ve uncovered that the requested “Excel feature” is just one approach. The real issue is the lack of automation for repetitive work, and the solution might offer far greater value than the user initially realized.

By integrating these five strategies, CEOs and founders can cut through assumptions, identify the **actual** problem, and prioritize solutions with the highest impact. This kind of clear, direct engagement—watching users at work, asking probing questions, and quantifying outcomes—turns abstract theory into tangible action steps, ensuring that every solution built aligns with real business needs.

Note: Always challenge direct ‘solutions’ (“I need this button”, “Give me an Excel”). Dig into the real ‘why’ behind the request (as in the Excel example). Ensure the investment targets the root problem, not just a superficial symptom.

5.5 Common Inefficiencies, Bad Practices, and How to Fix Them

Below is an expanded table to help CEOs or founders identify typical pain points (*inefficiencies*), the corresponding bad practices that cause them, the problems they create, and practical solutions. Each solution references methods like **OOPSI**, **Specification by Example**, and **User Story Mapping**—all designed to help teams collaborate continuously and reduce rework.

Pain Point	Bad Practice	Why it's a Problem	How to Fix it
Projects run over budget or timeline	Vague or insufficiently detailed requirements	Developers end up making incorrect assumptions, causing rework and delays	Document requirements as structured user stories, define clear acceptance criteria, and use examples. OOPSI helps break down outcomes vs. outputs and can reduce ambiguity.
Wasted resources on unneeded features	Developers build features misaligned with actual business needs	Time and money are spent on functions that don't solve real user or market problems	User Story Mapping ensures each story ties back to real user goals. Combine it with OOPSI to maintain clarity on what is truly needed. Validate every feature with potential users or stakeholders first.
Team conflicts and lack of synergy	Excluding the tech team from product discovery	Solutions might be infeasible or technically inefficient, causing internal friction	Involve developers early and continuously. Shared ownership of requirements fosters alignment. Behavior-Driven Development (BDD) and OOPSI create a "common language" that both teams can understand.
Low user satisfaction or adoption	Not validating with real users before development	You risk building something that nobody actually wants, wasting development cycles	Use prototypes, MVPs, and user feedback loops to confirm that proposed solutions solve real problems. Specification by Example can test critical scenarios before heavy coding.
Endless rework and confusion	Insufficient acceptance criteria or examples	Developers keep guessing expected behavior, leading to subpar implementation	Clearly define acceptance criteria using Specification by Example . Write user stories with "Given-When-Then" to avoid ambiguity. Regularly review them with stakeholders for quick feedback.
"Black hole" of misunderstandings between Product Discovery and Delivery	Treating Discovery → Delivery as a one-way handoff	Knowledge gaps grow when Product Discovery hands off incomplete or unclear requirements	Keep continuous collaboration throughout the SDLC. Pair OOPSI with BDD to maintain alignment, capturing outcomes and validating them with real examples before and during development.

Key Takeaways

- **Avoid one-way handoffs.** Continuous involvement from both business and development teams ensures alignment and reduces miscommunication.

- **Clarity beats assumptions.** Structured frameworks like OOPSI, BDD, and Specification by Example help minimize guesswork and bring focus to real user needs.
- **Prototype and validate.** The earlier you get user or stakeholder feedback, the less costly the rework.

By integrating these practices, CEOs and founders can mitigate wasted effort, reduce confusion, and focus the development team on delivering real business value.

Conclusion: Communication Drives Everything

Software development is not just about writing code; it is also about effectively managing resources, prioritizing tasks, and minimizing wasted effort. A well-structured SDLC (*Software Development Lifecycle*) helps balance speed, quality, and cost. However, this balance often fails when nobody is truly accountable for organizing the work between the business vision and the development team. Below, we focus on why a **Project Manager** is so pivotal in bridging this gap—even if in some organizations that person might also be called “Product Manager” or “Product Owner.” The key is: **Someone must own what will be done and why, and ensure the team understands how and when.**

Chapter 6: Project Management and Decision-Making

Software development is not just about writing code; it is also about managing resources effectively, prioritizing tasks, and minimizing wasted effort. A well-structured SDLC (*Software Development Lifecycle*) helps maintain the balance between speed, quality, and cost. However, this balance is often jeopardized by confusion around roles—particularly around who manages the project, who owns the product vision, and who implements the technical solutions.

Below, we will explore:

1. The difference between **Product Management** and **Project Management**.
2. How a backlog should be managed as a collaborative, structured tool.
3. The exponential cost of technical debt and why ROI considerations matter.
4. Scrum vs. Kanban (*and why mixing them poorly causes confusion*).
5. Risk management fundamentals.

6. The importance of understanding **shortcuts vs. starting small**.

6.1 Why Project Management Matters

In many teams, there is a notion of a “Product Owner” (*from Scrum*) or Product Manager who sets priorities based on user and business needs. Yet if there is no dedicated **Project Manager or PO**, the Developers often waste time trying to translate broad product goals into day-to-day tasks. It’s nobody’s “full-time job” to manage the backlog, plan realistic timelines, clarify requirements, or weigh short-term gains against long-term costs.

What Does a Project Manager Do?

- **Defines (or refines) the ‘what’ and ‘why’** – In a small team, the Project Manager might also own the product vision and user needs. In larger setups, a Product Manager handles the vision per product or feature, while a higher-level Project Manager ensures alignment across them — especially when coordinating multiple products within a suite.
- **Structures the ‘how’ and ‘when’** – Ensures tasks are prioritized, tracked, and assigned so the Dev Team can focus on building.
- **Manages risk, budgets, and timelines** – Evaluates how technical debt might affect ROI, or whether a given feature’s potential value justifies the cost.
- **Facilitates collaboration** – Developers don’t want to guess business value. The PM ensures everyone understands both business goals and technical constraints.

In practice, the size of the team and the complexity of the project determine how responsibilities are distributed. In a very small team, a single person might take on multiple roles—such as Product Manager, Project Manager, and even part of the development. If the scope is limited and the goals are clear, that setup can work just fine. However, as the organization grows or product lines diversify, it’s more common for each initiative to have its own Product Manager (*focused on the “what” and “why” of each feature*) and at least one Project Manager to coordinate planning, risk, and progress (*the “how” and “when”*). **The key is that someone must clearly own the orchestration of tasks and alignment with the business**, ensuring there are no gaps and that teams are not operating without direction.

6.2 The Backlog: Not a Dumping Ground

A properly managed backlog is the backbone of software delivery. It is **not** a catch-all for every half-formed idea or random ticket. Instead, each work item in the backlog should represent a well-defined investment with clear expected outcomes or ROI.

***Note:** See the backlog not as a to-do list, but as an investment portfolio. Require (or help define) a clear ‘why’ for each significant item (business value, expected ROI, mitigated risk) before it consumes the development team’s time (and your money).*

1. Clarity Before Adding Items

- A ticket with just a title—no context, no outcome—creates confusion.
- If only one person knows what the task is about, it can’t be delegated or estimated.

2. Shared, Collaborative Space

- The backlog isn’t a personal to-do list. The **Project Manager** reviews or refines new tasks, ensuring they align with business needs and have enough detail for the Dev Team to execute.
- Developers can propose technical tasks or refactors, but the PM confirms each item’s business value and scope before it becomes “official” backlog.

3. Separate ‘Draft’ or ‘Idea’ Board

- If people still need somewhere to jot quick ideas, keep that separate from the main “active” backlog.
- The Project Manager later decides whether an item is worth promoting into the main backlog (*by clarifying cost, ROI, or risk*).

This ensures that the backlog is a **collaborative artifact** reflecting current priorities, not just a disorganized pile of “maybe someday” tasks.

6.3 ROI and Technical Debt: The Cost Grows Exponentially

A huge part of project management is deciding how quickly to deliver vs. how well to build for the future. **Technical debt** describes the hidden cost of cutting corners now—debt that grows exponentially if you keep building on shaky foundations.

Note: *The decision to take ‘shortcuts’ (incur technical debt) to move faster today has financial consequences that grow exponentially tomorrow. This is a strategic decision you must understand and approve (not just a technical one), balancing short-term gain against future cost and slowness—especially in Type B systems.*

Real-World Example: Missing User Management UI

Imagine you have a system that supports multiple user permissions and roles. Instead of building a UI for admins to manage user access, you decide to “just write a script.” At first, that seems faster. But as you add new features, different roles, and more complex permission schemes, you keep running that script manually to match each scenario. Over time:

- **Developers** must replicate the script in their local environment (*their individual computer setup used for development*) to keep data in sync, making set-up cumbersome.
- **End-to-end tests** (*automated tests that simulate a full user journey through the application*) break or need extra hacks because there’s no unified, automated way to create users or assign roles in the product itself.
- **Refactors** become riskier; any new feature might accidentally break that script-based “internal logic,” forcing major rework when you finally build the UI.

This is how technical debt grows. Every new feature depends on that original hack, so fixing it later costs far more—and may break many things. In a **Type B** (*long-term, critical*) project, this kind of debt is rarely justified. In a quick **Type A** experiment, it might be acceptable—but only if you plan to throw it away.

Who Decides?

- **Developers** can assess the technical impact, but the **Project Manager** must weigh the business ROI. If skipping the UI saves two days but causes exponential complexity over six months, it’s probably not worth it—particularly for a large, stable project (*Type B*).
 - In a short-lived MVP (*Type A*), that quick hack might be okay, provided everyone understands it must be replaced if the concept succeeds.
-

6.4 Scrum vs. Kanban: Keep It Simple

Scrum and Kanban are two popular agile project management frameworks designed to control risk and manage workflow differently, not to dictate how developers code. It's important to remember:

- **Developers** don't pick Scrum or Kanban; the **Project Manager** (*or an equivalent lead*) chooses the methodology best suited to the **risk tolerance** and **nature of work**.
- **Scrum** is time-boxed (*sprints*), good for innovation with uncertain goals and for teams comfortable with planning in fixed intervals.
- **Kanban** is a continuous flow, better for teams dealing with frequent priority changes or routine tasks that arrive unpredictably.

Advantages vs. Disadvantages (*Briefly*)

- Scrum can help teams stay focused on an "Outcome" each sprint, but changing priorities mid-sprint is harder.
- Kanban excels when tasks vary widely or arrive at random times, but it can lack the structured "risk window" that Scrum provides.
- Hybridizing them often dilutes the benefits of each.

The main point: **Choose one approach** and run it properly instead of making it a developer's problem. Over time, if your project style changes, you can switch frameworks (*or keep it very lightweight if it's a small team*). The real question is who is managing the risk, how often you re-plan, and how you measure progress.

Teams often struggle to choose between Scrum and Kanban. **Both** are methods of managing workflow and mitigating risk, but their core philosophies differ:

Factor	Scrum	Kanban
Focus	Delivering outcomes (<i>value</i>) in short, time-boxed increments	Maximizing continuous throughput (<i>the amount of valuable work completed over a period of time</i>) of discrete tasks
Risk Approach	Risk per sprint (2-3 weeks, for example)	Risk per task (<i>small batches, continuous flow</i>)
Ideal Scenarios	Innovating with uncertain goals or big changes every sprint	Handling routine tasks or incoming tickets with frequent priority shifts
Ownership	Product Owner owns the backlog; dev team commits to sprints	Team (or PM) manages a continuous backlog flow
Ceremonies	Sprint planning, daily stand-ups, sprint review, retrospective	Continuous pull of tasks, WIP limits, possibly daily stand-ups if needed

Kanban often uses WIP limits (*Work In Progress limits – restricting the number of tasks the team works on simultaneously*) to maintain smooth flow and quickly identify bottlenecks.

Why Not a Hybrid?

Mixing Scrum's ceremonies (*like sprint reviews*) with Kanban's continuous flow often **dilutes the benefits** of both. It usually signals that:

- The team lacks a clear understanding of either method.
- The organization wants the structure of Scrum but the flexibility of Kanban, leading to **no real structure** at all.
- Roles often become blurred—for example, you might see someone labeled a "Product Owner" running a Kanban process, or a Scrum Master trying to facilitate delivery in a team where priorities change daily. In these cases, the mismatch between the role and the methodology creates confusion and undermines

accountability.

In reality, the **Project Manager** (*or in some cases, the PO*) should pick the methodology based on the **risk** they can tolerate and the **type of work** being done. If the risk is time-boxed and outcomes are uncertain, Scrum might fit. If tasks are well-understood, short, and constant in priority changes, Kanban often makes more sense. But jumping arbitrarily between them can create confusion and degrade accountability.

6.5 Risk Management in Software Development

Regardless of methodology, software projects carry inherent uncertainty. A **Project Manager** ensures that risks (*technical or business*) are systematically identified and mitigated.

1. Iterative Deliveries

- Whether using sprints or continuous flow, delivering in small increments uncovers problems early.
- This aligns with ideas from **Continuous Delivery** (*Jez Humble, David Farley*).

2. Automation and Monitoring

- Automated tests, CI/CD pipelines, and real-time metrics reduce the risk of human error.
- If something breaks in production, you catch it faster, mitigating cost.

3. Clear Accountability

- A Project Manager tracks open tasks, ensures the backlog is updated, and checks whether something has been stuck in “in progress” for too long.
 - The Dev Team flags potential issues (*e.g., “We’re incurring high technical debt on X,” or “This approach might be too slow”*), so the PM can weigh the trade-offs with business stakeholders.
-

6.6 Shortcuts vs. Starting Small

There’s a world of difference between **starting small**—implementing a minimal but legitimate version—and **taking shortcuts**—building something hacky that undermines your project’s foundation.

Starting small (*for example, launching a basic but well-built version*) is a smart strategy for validating ideas and iterating efficiently. By deliberately limiting scope without sacrificing proper design, you can collect real user feedback and evolve the product incrementally. In contrast, taking shortcuts (*such as faking core functionality instead of properly building it*) is a ticking time bomb. While it might feel faster at first, the hidden technical debt can explode later, requiring expensive refactoring and delaying critical features.

Make sure both you and your team understand and apply this crucial distinction—so you can embrace the benefits of starting small without falling into the trap of harmful shortcuts.

- **Starting Small**

- Example: Implement a user list without pagination (*the 'next page' / 'previous page' controls typically used for long lists*) in the first release. If you see real user demand for bigger lists, add pagination later.
- This approach is intentionally minimal but still structured enough to avoid major rewrites. It's not a “hack”—it's a genuine first slice.

- **Shortcuts**

- Example: Hard-coding user data (*embedding data directly into the code instead of loading it dynamically from a database or API*) in the front end, skipping the normal data flow entirely.
- Faster at first, but a massive headache later. Once your project grows, you must rip out that hard-coded logic and rebuild from scratch.

In **Type B** (*stable, long-term*) projects, such shortcuts are rarely a good idea. In **Type A** (*quick tests or MVPs*), you might accept more hacks for speed, but only if you have no intention of reusing that code in a final product. A Project Manager who understands the difference can protect your team from dangerous “tech debt bombs” while still allowing valid experiments.

Conclusion: Who's Actually Driving the Project?

In any software project, someone must decide which tasks are worth doing, how they're prioritized, and what level of quality or speed is acceptable. That person—call them a Project Manager, Product Manager, or Product Owner—ensures the team stays aligned on outcomes, manages risk, and avoids drowning in chaos.

- A **well-maintained backlog** with fully understood tasks sets the stage for efficient delivery.

- **Technical debt** grows exponentially, so you must decide early if it's worth risking future budget for quick gains today.
- **Scrum vs. Kanban** is a decision about **risk management**—not a developer preference.
- **Shortcuts** can create crippling debt; **starting small** remains a better strategy for building iteratively without huge rework down the line.

By assigning **Project Management** responsibilities clearly—especially in larger teams—organizations reduce friction, keep ROI in focus, and ensure developers can do what they do best: build high-quality software that delivers real value.

Chapter 7: The Step-by-Step Software Development Process

In this chapter, we'll walk through the entire process from the perspective of a CEO or founder, highlighting the importance of aligning your Software Development Life Cycle (SDLC) with your organization's goals—whether you're working on a quick proof of concept (**Type A Project**) or a robust, production-ready system (**Type B Project**).

7.1 From Ticket Creation to Final Delivery

A structured workflow in the SDLC typically follows these steps:

1. Requirement Creation (*Including Cost, ROI, and User Value*)

- **Business Value & ROI:** Before writing a single line of code, make sure you know why you're creating a feature in the first place. What does it cost, and how does it generate a return on investment?
- **Specification by Example (*Early Alignment*):** Techniques like **Specification by Example** (*Gojko Adzic*) are best applied when defining requirements so the team can discuss concrete examples right away.
- **User Value & Use Cases:** Tools such as **User Story Mapping** (*Jeff Patton*) or **OOPSI** help visualize user journeys. This ensures each requirement delivers something worthwhile to end users.

2. Prioritization and Planning

- **Roadmap for Type A vs. Type B:** Decide if you're aiming for fast validation (*Type A*) or a more stable system (*Type B*).

- **Aligning Goals with Stakeholders:** Show investors or executives how each requirement ties back to the company's objectives or ROI.

3. Technical Design and Approval

- **Balancing Speed and Structure:** If it's a Type A proof of concept (*POC – An experiment is usually smaller than an MVP, focused on testing technical feasibility or a single core idea*), minimal documentation might be enough. If it's Type B, you'll likely need a more formal approach, possibly **Clean Architecture** (*Robert C. Martin*) for long-term maintainability.

4. Development and Testing

- **Varying Levels of Rigor:** Type A projects might be okay with fewer tests, while Type B often demands a robust suite of unit tests (*checks on small code pieces*), integration tests (*verifying that different parts of the software work correctly together*), and acceptance tests.
- **Maintaining Business Focus:** Even in development, keep an eye on the strategic goals behind each feature.

5. Deployment and Monitoring

- **Feedback Over Perfection (Type A):** For quick prototypes, you might not need a complicated pipeline or extensive monitoring; direct user feedback might suffice.
- **Reliability and Scalability (Type B):** A formal Continuous Delivery pipeline and monitoring are critical for production-grade products, helping validate that each new feature meets expectations.

6. Feedback and Optimization

- **Cross-Project Learning:** A Type A project can inform a future Type B iteration if the concept proves worthwhile.
- **ROI Confirmation:** For a CEO or founder, validating the return on investment is essential—if the results aren't there, pivot or drop the feature and move on.

7.1.1 Starting with Five Essential Questions

Before creating any ticket or diving into development, it's critical to pause and evaluate the context in which a new feature (*or project*) will live. Below are five questions that guide this initial decision-making:

Note: *Make sure these 5 questions (Type A/B? Validated? Real problem? ROI? Risk/Budget?) are answered before the team invests significant time*

in any new initiative. This is your essential strategic filter to avoid wasting valuable resources.

1. What Type of Project Is This?

Are we looking at a quick, experimental initiative (*Type A*) or a more robust, long-term solution (*Type B*)? Defining the project type sets the tone for everything else: the level of rigor in testing, documentation, architecture, and budget allocation.

2. Is the Idea Validated?

Do we have any evidence—user interviews, small-scale tests, or market research—indicating that this feature is genuinely needed or wanted? If not, it may be better to run a rapid validation exercise before fully committing resources.

3. Real Problem or Just an Instruction?

Use methods like the “5 Whys” to identify if you’re solving the root cause rather than chasing a superficial request. For example, if someone says “Add a payment button,” dig deeper to confirm that checkout friction is indeed the issue, not something else.

4. What’s the Potential ROI or Business Impact?

If you’re targeting a 10% improvement in conversions, what does that mean in actual revenue or savings? Tying features to measurable outcomes helps prioritize and justify investment.

5. What Are the Risk and Budget Constraints?

Are we dealing with high uncertainty and minimal runway (*favoring a Type A approach*)? Or do we have a stable, well-funded project that justifies a more rigorous Type B style from the start?

These five questions frame the discussion around goals, feasibility, and potential payoff. Once you have answers—and especially if you confirm the idea is worth pursuing—you’re ready to move on to the detailed process of bringing the feature to life.

7.1.2 Step-by-Step Flow

Say you’re adding a “Payment Button” feature—the following steps outline how you might proceed after clarifying the five questions above. For brevity, we’ll label each stage from A through H:

1. Step A: Triage Basic Questions

- **Purpose:** Double-check the five essentials above. Decide on project type (A or B), confirm validation, and assess risk/ROI.
- **Example:** If you determined that the “Payment Button” is partially validated, but some uncertainty remains, you might treat it as Type A for an initial pilot.

2. Step B: Create Ticket / Work Item

- **Purpose:** Formally record the requirement (e.g., in Jira or Trello).
- **Actions:**
 - Write the requirement as a user story:
“As a user, I want a Payment Button so I can complete my checkout faster.”
 - Add acceptance criteria (e.g., *successful transaction, receipt emailed to user, etc.*).
 - Include the target ROI (e.g., *“We aim for a 10% increase in completed purchases”*).

3. Step C: Refinement & Approval

- **Purpose:** Discuss feasibility, confirm scope, finalize timeline and success metrics.
- **Actions:**
 - Tech leads or architects review the requirement for technical complexity.
 - CEO/founder might weigh in on final ROI expectations or budget/time constraints.

4. Step D: Architecture & Design

- **Purpose:** Define how everything fits together. Even if you don’t create a full diagram, sketch key workflows.
- **Actions:**
 - **Flow:** “User clicks button → Payment is processed → Confirmation is displayed → Receipt is sent.”

- **Sequence:** “Frontend calls backend → Backend hits payment gateway → DB updates the order → Response to user.”
- **Infrastructure:** If Type B, you might design a more robust setup (e.g., *using AWS Lambdas, message queues, or a microservice approach*).

5. Step E: Development & Testing

- **Purpose:** Implement the feature, balancing speed (*Type A*) vs. reliability (*Type B*).
- **Actions:**
 - **Type A:** Possibly minimal testing, focusing on user feedback.
 - **Type B:** Full suite of unit/integration tests. Could involve formal code reviews, peer testing, and documentation.

6. Step F: Deploy & Monitor

- **Purpose:** Release the feature and keep an eye on behavior in the real world.
- **Actions:**
 - **Type A:** Possibly just a manual deployment or a lightweight script; monitor via direct user feedback.
 - **Type B:** Use a CI/CD pipeline (e.g., *Jenkins, GitHub Actions*), real-time logs, and dashboards to catch errors or performance bottlenecks.

7. Step G: Gather Feedback

- **Purpose:** Check how users respond—either informally (*Type A*) or via metrics (*Type B*).
- **Actions:**
 - Collect user comments or measure conversion rates post-deployment.
 - Identify any friction or errors that users report.

8. Step H: Optimization & ROI Check

- **Purpose:** Validate whether the feature meets expectations; if not, pivot or improve.

- **Actions:**
 - Compare results against the ROI target: **Did we hit that 10% lift in completed purchases?**
 - If still Type A but now validated, consider upgrading it into a Type B approach with better automation, testing, and monitoring.
 - If results aren't promising, evaluate whether to remove or pivot the feature entirely.

By following steps A through H—and by continually referencing the five key questions from **7.1.1**—you ensure that each new feature is both strategically justified and well-executed. For a CEO or founder, these checkpoints help maintain focus on business outcomes rather than just technical deliverables, guiding your team toward meaningful, value-driven software development.

7.2 Flowchart, Sequence Diagram, and Infrastructure Diagram

Designing software effectively requires visual representations that clarify functionality and data flow. Here are three fundamental diagrams to consider:

1. Flow Diagram (*Flowchart*)

- a. **Purpose:** Offers an overview of how a user will interact with the functionality—navigating pages, clicking buttons, receiving emails, etc.
- b. **Why CEOs Care:** Helps non-technical stakeholders see the user journey. This is essential for verifying that you're building the right feature in a way users will actually engage with.

2. Sequence Diagram

- a. **Purpose:** Illustrates the step-by-step **technical** interactions among different system components (e.g., *user interface*, *server*, *database*) required to fulfill a specific user action. When the user clicks a button in the app, what calls are made to the REST API (*a standard way for different software systems or components to communicate with each other over the web*), and what data flows occur behind the scenes?
- b. **Why CEOs Care:** Demonstrates, in simple terms, how each technical component delivers on user expectations, preventing confusion over how parts fit together.

3. Infrastructure Diagram

- a. **Purpose:** Shows the systems and services you'll need, such as databases, message queues (*systems used to manage tasks asynchronously, often improving system responsiveness and reliability, e.g., AWS SQS*), or serverless functions (*e.g., AWS Lambdas*).
 - b. **Why CEOs Care:** Every additional infrastructure component can mean added cost and operational complexity. An infrastructure diagram helps you see exactly where and why you're investing.
-

7.3 Ensuring Software Quality: A Strategic Perspective

While automated testing, Continuous Integration (CI), and Continuous Deployment (CD) are common technical practices, a CEO or founder needs to consider **why** and **when** to invest heavily in these areas:

1. Automated Testing

- a. **Type A Projects:** Speed is key. Detailed tests could slow the feedback loop, so minimal testing might be acceptable. The goal is to validate ideas quickly rather than ensure flawless stability.
- b. **Type B Projects:** Quality and reliability matter. Automated tests are vital to prevent regressions (*old bugs reappearing or previously working features breaking after a change*) and maintain user trust.

Note: *For your critical systems (Type B), investing in quality (automated tests, CI/CD pipelines) is not an optional extra or a purely technical cost. It's a direct investment in your product's reliability, future delivery speed, customer satisfaction, and the protection of your revenue and reputation.*

2. Continuous Integration / Continuous Deployment

- a. **Type A Projects:** Rapid experimentation may not warrant a robust pipeline if the product is purely exploratory. Manual deployment or basic packaging could be enough.
- b. **Type B Projects:** A well-established CI/CD pipeline (*as described in **Accelerate**, by Nicole Forsgren, Jez Humble, and Gene Kim*) ensures both speed and quality, automating repetitive tasks and quickly surfacing issues.

3. Monitoring and Alerts

- a. **Type A Projects:** Frequent system errors might not be a deal-breaker. Direct human feedback is often more important than automated alerts for early prototypes.
 - b. **Type B Projects:** Monitoring is essential to detect performance or reliability issues and to track usage patterns. Alerting systems can flag critical problems before they affect large numbers of users—and thus protect ROI.
-

7.4 Scaling the Development Team

For a CEO or founder, scaling often means transitioning from a small, scrappy experiment (*Type A*) to a more structured, sustainable product (*Type B*):

- **Process Overhaul:** If your Type A project shows promise, you'll need to **introduce formal processes** (*Kanban* or *Scrum*), a **centralized backlog**, and clearer workflows. Simple Slack messages to a single developer won't cut it as you onboard more people.

***Note:** Scaling a development team isn't just about hiring more engineers. It requires conscious investment in formalizing processes, improving documentation, and structuring communication. Without this foundation, adding people often slows down delivery rather than speeding it up.*

- **Documentation and Knowledge Sharing**
- **Centralize Information:** Move from tribal knowledge to a shared wiki or internal documentation system.
- **Establish Standards:** Code style guides, code review guidelines, and decision logs help maintain consistency and quality across a growing team.
- **Team Coordination and Specialized Roles**
- **Infrastructure/Quality Specialists:** As the product matures, you may need experts in DevOps, QA, or security.
- **Formal Retrospectives and Reviews:** Borrow ideas from **50 Quick Ideas for Improving Your Retrospectives** to continually refine processes as your team grows.
- **Managing the Transition**
- **From Idea to Product:** Shifting a Type A experimental mindset to Type B operational rigor requires cultural changes. Faster releases might slow down to ensure stability,

but the potential payoff in user satisfaction and longevity is worth it.

- **Long-Term Sustainability:** Ultimately, a robust SDLC aligned with your business model will enable you to scale confidently, balancing innovation with reliability.
-

7.5 Cultural Shifts: Transitioning from Type A to Type B

When teams have spent prolonged periods operating in a **Type A** mode—constantly experimenting, seeking quick market feedback, and validating ideas—they often develop habits and mindsets optimized for **rapid iteration over robust execution**. As soon as their company (*and product*) hits a growth phase that requires a **Type B** approach, these ingrained Type A habits can create friction. This transition is more than just a change in technical processes; it represents a **deep cultural shift** for both developers and the broader organization.

7.5.1 Why the Cultural Clash Occurs

1. Different Goals and Standards

- **Type A (Experimentation):** The goal is to validate an idea quickly, often by building minimal solutions or prototypes. Frequent pivots, minimal testing, and ad-hoc processes are often acceptable if they lead to faster feedback.
- **Type B (System):** Focuses on creating a **scalable, reliable product**, which requires more thorough design, robust testing, and stricter attention to non-functional requirements (*e.g., performance, security, maintainability*).

2. Teams accustomed to Type A can see these new standards as “slowing down,” while those pushing for Type B view them as **crucial investments** for long-term stability.

3. Team Mindset

- In prolonged Type A modes, the development team treats **coding itself as part of Product Discovery**—they build, test, and measure directly in the market.
- When shifting to Type B, the team needs to **separate** Product Discovery (*identifying the real user problems and solutions*) from **system-building** (*implementing robust features*). This often means **relearning** how to gather insights (*e.g., asking “5 Whys,” analyzing user behavior*) before jumping to code, and writing significantly more tests.

4. From Quick Hacks to Sustainable Solutions

- **Type A** solutions can be “just enough” to validate a hypothesis. These same solutions, if simply carried forward, rarely scale.
 - **Type B** ultimately might not even reuse the original prototype’s code. Instead, it incorporates the *learnings* from Type A but implements them in a more **comprehensive, future-proof** way.
5. This is a cultural shock: developers who are used to “fix it now, move on tomorrow” must adjust to “build it for the long haul.”

7.5.2 Impact on the Development Team

- **Shift in Coding Practices**

In Type A, writing minimal or no tests might be acceptable, especially if you’re unsure the feature will exist next month. Under Type B, **tests become non-negotiable**, code reviews more structured, and architecture decisions more deliberate.

- **Change in Speed Expectations**

- **Velocity** can feel slower because you’re investing in pipelines, automated tests, documentation, and robust integrations.
- Developers may feel frustrated if they’ve **valued shipping speed** above all else. Suddenly, they must prioritize code quality, reusability, and reliability.

- **Reevaluating Discovery**

Developers who once discovered user problems by coding on-the-fly must adapt to more formal **Product Discovery** steps. For them, validating doesn’t just mean “Can we build it quickly?” but rather “Have we aligned the problem and solution thoroughly before coding?”

- **Ownership and Accountability**

A Type B system involves higher stakes: real customers, enterprise-level expectations, potentially mission-critical processes. Failing under these conditions is more damaging than in an experimental environment, pushing developers to care **not just about building** but also about **maintaining** and **scaling**.

7.5.3 Impact on Product & Business Teams

- **Changing Expectations**

When a business has been in rapid market-fit mode (*Type A*), product managers and

founders get used to near-instant iterations. In a Type B project, they must recognize that **quality, reliability, and regulatory compliance** can't always be delivered at breakneck speed.

- **Investment in Non-Functional Requirements**

- Building a robust product involves intangible work: setting up **CI/CD pipelines**, ensuring **monitoring and alerting**, factoring in **architecture** that supports multiple clients, etc.
- This work may not produce immediate visible output but is essential for stability and user satisfaction.

- **From “Solution-Focused” to “System-Focused”**

- Type A is often about delivering “just enough” for each new client or user group.
- Type B requires building a **product** that is widely deployable, maintainable, and sells at scale to multiple clients—rather than piling on dozens of piecemeal features that can't integrate seamlessly.

- **Alignment with Long-Term Strategy**

- Leadership must shift from seeing immediate, iterative features as the sole metric of progress to **valuing robust infrastructure, architectural planning, and methodical roadmaps**. This implies budgeting for work that doesn't directly translate into immediate user-facing features but is critical for long-term success.

7.5.4 Navigating the Cultural Transition

1. **Acknowledge the New Reality**

- Communicate early that **velocity may decrease initially** but the stability and future payoff will be worth it.
- Encourage open discussions about **why** the shift is necessary: scaling the user base, meeting enterprise demands, ensuring reliability, etc.

2. **Invest in Training & Support**

- If developers have never implemented advanced CI/CD, robust testing suites, or architectural patterns, **provide the resources** they need to succeed (*mentors, training, code reviews*).

- Product managers and business stakeholders may need refreshers on **non-functional requirements** and how they directly affect user retention, brand reputation, and revenue.

3. Define Clear Phases

- Separate **Product Discovery** from **System-Building**. Clarify that Type A experimentation remains valid when exploring new ideas, but once validated, **development transitions** into Type B with stricter guidelines and standards.
- This dual-track approach helps teams see that Type A and Type B are **complementary** rather than conflicting.

4. Emphasize Transparency in Planning

- Clearly distinguish tasks aimed at validating a hypothesis (*Type A*) from those aimed at **robust implementation** (*Type B*).
- Show how each step contributes to overarching business goals: “We’ll test feasibility (*A*), then incorporate the lessons into a sustainable platform (*B*).”

5. Celebrate Wins Differently

- In Type A, success was often measured by “How fast did we get that out?” or “How quickly did we pivot?”
- In Type B, success may be measured by “How stable is our system under load?” or “Did we meet our reliability SLAs?”
- Reinforce this shift in priorities so teams know what **quality metrics** to value.

7.5.5 Addressing Skill Atrophy When Transitioning from Type A to Type B

One overlooked aspect of cultural change is the skill set required for teams that have long operated in Type A mode. In Type A projects (*experimental or MVP*), developers typically focus on scripting or “just enough” coding to get an immediate solution working. They rarely employ systematic design patterns, robust architectures, or formal testing processes. Over time, this can lead to skill atrophy—an erosion of the developer’s ability to design and implement systems at scale because their primary mindset is geared toward quick fixes and minimal overhead.

Why Skills Deteriorate in Extended Type A Environments

- **No Systematic Patterns.** Developers often code top-to-bottom solutions with minimal abstraction, ignoring established patterns like CQRS, microservices, or monolithic modular architectures. While effective for quick experiments, this mindset can undermine the ability to think in terms of bounded contexts (see *Domain-Driven*

Design by Eric Evans) or modular design (*Clean Architecture by Robert C. Martin*).

- **Limited Testing and Architecture.** In MVPs, formal tests or robust architectural discussions are often seen as optional. Over time, developers may lose familiarity with TDD, BDD, or basic unit-testing practices (*Refactoring by Martin Fowler and Growing Object-Oriented Software Guided by Tests by Steve Freeman and Nat Pryce offer excellent guidance here*).
- **Focus on Immediate Results.** Constantly “solving the problem fast” means there is no habit of planning for scalability or maintainability. Important topics like optimistic vs. pessimistic locking, concurrency control, or advanced data modeling get sidelined in favor of immediate delivery.

Conway’s Law and Communication Patterns

In **Team Topologies** (*Matthew Skelton and Manuel Pais*), Conway’s Law is highlighted: the structure of your organization and communication flows shapes (*and is shaped by*) your software architecture. When developers are used to Type A “script-first” approaches, they also carry that communication style—fast, ad-hoc, and minimal design discussions—into Type B initiatives. This mismatch can hinder the adoption of more formal architecture or complex design strategies because the cultural communication patterns resist the depth and rigor required.

Why This Matters in Scaling (*Type B*)

- **Architecture Requirements.** Building a scalable system (*Type B*) demands patterns, layering, and consideration of non-functional requirements (*performance, security, maintainability*). A team lacking these skills may deliver fragile, hard-to-scale code.
- **Quality and Testing.** Type B systems generally require thorough testing, continuous integration, and reliability measures. Without prior exposure or training, developers may struggle to adapt to automated pipelines, coverage metrics, or advanced QA practices.
- **Long-Term Viability.** If the same Type A-heavy team attempts to transition but keeps the same “just hack it” mindset, the result can be a codebase riddled with shortcuts, technical debt, and poor design, ultimately failing the needs of an enterprise-ready system.

Practical Steps to Bridge the Gap

1. Selective Hiring or Upskilling

Bring in senior developers or architects with proven Type B experience (e.g., *scaling e-commerce systems, distributed architectures*) to mentor the existing team. Alternatively, offer internal training on topics like Domain-Driven Design or

microservices best practices.

2. Pairing and Mentoring

Pair Type A developers with Type B veterans for architectural reviews, code reviews, and test design. This ongoing collaboration accelerates learning and changes coding habits.

3. Structured Knowledge Transfer

Host internal workshops on design patterns, database transactions, concurrency control, and QA fundamentals. Reference **Clean Code** and **Clean Architecture** (*Robert C. Martin*) or **Refactoring** (*Martin Fowler*) to build a common vocabulary and skill set.

4. Gradual Adoption of Patterns

Introduce new architectural practices incrementally. For instance, start with well-defined boundaries in a monolith before leaping to microservices. Emphasize Bounded Contexts from Domain-Driven Design if the domain is large or complex.

5. Effective Retrospectives and Team Communication

Use retrospectives (see *50 Quick Ideas for Improving Your Retrospectives*) to discuss where rapid, ad-hoc practices are undermining scalability. Encourage open dialogue (*Crucial Conversations*) so the team feels safe highlighting skill gaps or confusion around advanced designs.

6. Align Incentives and Metrics

Shift the success metrics from “We shipped fast” to “We shipped fast and the system remained stable.” Reward code clarity and maintainability as much as immediate delivery. Over time, this reinforces more sustainable development practices.

By consciously recognizing how prolonged Type A work can distort or weaken a developer’s architectural and design skill set, you can plan the necessary training, mentoring, and hiring strategies. Teams that make this transition effectively retain the agility of quick discovery while meeting the rigorous demands of a scalable Type B environment. They also embrace Conway’s Law constructively—shaping communication patterns and team structures that naturally foster robust architectures.

Conclusion: Aligning Your SDLC, Your Culture, and Your Business Needs

Whether you’re **validating an untested concept (Type A)** or **building a mission-critical system (Type B)**, always connect your process—and your mindset—to the project’s true

objectives. A culture centered on rapid experimentation can quickly deliver invaluable insights when searching for market fit; yet, transitioning to Type B requires deeper investments in quality, architecture, and long-term sustainability.

Key Takeaways for CEOs and Founders

1. Ask the Right Questions First

Determine the project type (*A or B*), validate the idea, ensure there's a real problem to solve, and define the ROI and risk budget.

2. Decide on Speed vs. Rigorous Process

Type A emphasizes rapid feedback and minimal overhead, while Type B demands structured pipelines, robust testing, and a focus on maintainability.

3. Track ROI and Business Impact Continuously

Even experimental features need a potential payoff. Keep your eyes on metrics—revenue, conversion rates, or other strategic goals—to confirm real-world value.

4. Scale Up Processes When Needed

As soon as you confirm market fit and user adoption, evolve from quick prototypes to a more formal, stable system. That shift will require cultural changes across both development and product teams.

Embracing the Cultural Shift for Long-Term Growth

As your organization transitions from **rapid, reactive development** (*Type A*) to **structured, scalable products** (*Type B*):

- **Developers** must learn to separate discovery from delivery and be willing to invest in the robustness of code that outlives short experimentation cycles.
- **Product/Business** must appreciate that Type B inevitably demands more planning, processes, and architecture work, understanding that improved stability and reliability are worth the initial slowdown.

By **proactively addressing these cultural hurdles**, your company can seamlessly move from a startup mindset—thriving on swift prototypes and pivots—to a more mature stage capable of delivering dependable, scalable products. Crucially, you retain the flexibility to

keep experimenting when needed without sacrificing the core strength and sustainability of your system.

By blending these steps, questions, and strategic decisions, you'll ensure that every ticket created—and every feature you invest in—directly serves the **broader mission of your business**, aligning with both **short-term discovery** and **long-term success**.

From Theory to Action

What We've Learned

Chapter 1: The Software Development Machine

- Your SDLC is like a fine-tuned machine where every part should align with the outcome you want.
- There are two main types of projects:
 - Type A Projects (*MVPs/Experimental*) → Fast validation with minimal resources.
 - Type B Projects (*Scalable Systems*) → Robust architecture for long-term sustainability.
- Choosing the right project type (*A or B*) before development begins prevents costly rework and technical debt.
- Trying to blend Type A and Type B in the same codebase usually delivers the worst of both worlds: technical fragility with no speed advantage.
- Starting small and cutting corners are not the same. One is strategic and intentional; the other creates exponential technical debt.

Chapter 2: Types of Projects

- Type A and Type B require different roles, standards, and mindsets—not just different code.
- The cost of choosing the wrong approach up front grows exponentially over time.
- A successful MVP should trigger a deliberate transition to Type B, not evolve accidentally.

- Refactoring from A to B incrementally is far better than rebuilding from scratch. You retain domain knowledge and reduce estimation risk.

Chapter 3: Discovery Meets Delivery

- Product Discovery and Software Delivery are not sequential or parallel—they're collaborative.
- They work as two gears in constant interaction: one feeds clarity, the other execution.
- Every feature should be approached with five essential questions: What type of project is it? Is the idea validated? Are we solving the right problem? What's the ROI? What are the constraints?
- Identifying the real problem is a problem in itself. Frameworks like OOPSI, Impact Mapping, Specification by Example, and BDD help uncover root causes and align teams.
- Not everything discovered needs to become software. Sometimes the best solution is a process change or operational tweak.

Chapter 4: The Team Orchestration and Structure

- Team structure should change based on the type of project—not the people involved.
- You don't need two teams for two types of projects. People can operate in different modes depending on the project's needs.
- Culture plays a critical role in the success or failure of a transition from Type A to Type B. Habits built for speed often clash with the rigor needed for scale.
- A well-configured team (*with clarity of roles*) is more important than pure technical expertise.
- Cultural misalignment is a common source of project risk.

Chapter 5: Coordinating Through Communication

- Backlogs are not dumping grounds. Every task represents an investment and should have a defined ROI.
- Continuous collaboration is better than handoffs—requirements should evolve with feedback and context.

- Use concrete examples (*Specification by Example*, *BDD*) and structured communication (*OOPSI*, *user stories*) to reduce ambiguity.
- Techniques like the 5 Whys, impact mapping, and visual validation help you define the right problem and the expected outcome.
- Clear problem definition directly impacts development efficiency and final value.

Chapter 6: Project Management and Decision-Making

- Project management is not optional. Someone must orchestrate how and when things get built, and ensure delivery aligns with business strategy.
- Technical debt is not a one-time event—it accrues interest exponentially. It should be approached strategically and consciously.
- In Type B projects, technical debt is rarely acceptable. In Type A, if taken, it must come with the expectation of discarding the project.
- The backlog should be treated like an investment portfolio, not a wish list.
- Choosing Scrum or Kanban is not a developer's preference—it's a strategic decision based on acceptable risk and delivery rhythm.
- Mixing methodologies (*Scrum + Kanban*) without understanding them usually means you're not applying either well.

Chapter 7: The Step-by-Step Software Development Process

- A structured flow—from idea validation to monitoring—ensures alignment and delivery quality.
- Tools like flowcharts, sequence diagrams, and infrastructure diagrams are essential, especially in Type B projects, to clarify scope, architecture, and responsibilities.
- Scaling a team isn't just about hiring—it's about adding processes, documentation, and structure.
- Methodologies and roles must align. When they don't, chaos follows.
- Each deployment should close the loop with measurable feedback: did it create value? If not, pivot or discard.
- The skill set required for Type A and Type B projects can be radically different. You either train your team consciously through cultural and technical transitions—or have a strategy to hire people who already possess the skills needed to scale into Type B

systems.

Applying the Framework: Key Questions for CEOs & Founders

This section addresses some common, high-level questions that CEOs and founders may have after reading this book. The answers synthesize concepts from various chapters to provide a strategic perspective for leadership.

1. How do I *really* decide between a Type A (*MVP*) and Type B (*System*) approach, especially if an idea feels like it's in between?

- **Focus on the Primary Goal & Risk:** The core differentiator is your immediate objective. Are you primarily trying to **learn** and validate a core hypothesis with minimal investment (*Type A*), accepting high uncertainty about the final product? Or are you building something you **know** needs to be reliable and scalable, even if starting small (*Type B*), justifying higher initial investment to reduce future risk and rework?
- **It's a Spectrum, but Avoid the Hybrid Trap:** While there's a spectrum, consciously lean towards one primary approach for a given initiative. Trying to be both cheap/fast *and* robust/scalable **in the same codebase** usually fails. If you need both aspects, consider the dual-track approach (*running separate A and B efforts*) or sequence them: validate with A, then invest in B.
- **Involve Key Leads:** This is a strategic decision made **with** input from your Product and Tech leads, weighing market uncertainty against technical requirements and long-term vision.

2. My team talks about "Technical Debt." How do I know when it's acceptable, when we *must* invest in fixing it, and how can I gauge the ROI of that investment?

- **Debt is Strategic, Not Just Technical:** Think of technical debt like financial debt. Sometimes taking a small, calculated shortcut (*debt*) for significant short-term gain (*like validating an idea quickly in Type A*) **might** be acceptable, **if** you have a clear plan to "pay it back" (*refactor*) soon. Chronic debt in a core system (*Type B*) cripples future speed and increases risk exponentially.
- **ROI is About Future Enablement:** Quantifying the exact ROI of fixing debt is hard. Frame it in terms of **impact**: "Fixing this allows us to launch new features X% faster," "Reduces critical production bugs by Y%," or "Unlocks market Z by enabling scalability." Your Tech Lead should help articulate this impact, even if precise numbers are elusive.
- **Prioritize Based on Pain & Strategy:** Address debt that causes the most friction **now** (*slows down critical features, causes frequent outages*) or blocks strategic

goals. Use the backlog as an investment portfolio – paying down debt is an investment in future velocity and stability.

3. We started with an MVP (*Type A*) and it's successful! Now what? What's the realistic cost and process for scaling it to a robust (*Type B*) system?

- **Acknowledge the Investment:** Transitioning from A to B is a significant undertaking, not a minor tweak. It **will** require dedicated time and resources – potentially comparable to or even exceeding the initial MVP cost, depending on the accumulated debt.
- **Refactoring is Necessary:** The core activity is refactoring – improving the internal structure, quality, and architecture – alongside potentially adding more robust infrastructure, testing, and monitoring. It's rarely wise to scrap the MVP entirely, as it contains valuable validated learnings and domain knowledge.
- **Plan the Transition:** Treat the transition as a formal (*likely Type B*) project. Allocate budget, define clear goals (*scalability targets, reliability needs*), and adjust team processes. This is where formal quality practices (*testing, CI/CD*) become critical investments.

4. How much Product Discovery is 'enough'? How do we balance the need for validation with the pressure to deliver features quickly?

- **Discovery Reduces Risk, It Doesn't Eliminate Speed:** Frame Discovery not as a delay, but as **risk reduction**. Skipping it often leads to building the wrong thing faster, which is far more wasteful.
- **Focus on Riskiest Assumptions:** Prioritize validating the assumptions that, if wrong, would cause the biggest problems or waste the most resources. Use lightweight methods (*interviews, prototypes, simple tests*) first.
- **Timebox and Iterate:** Allocate specific, limited time for discovery cycles (*"Let's spend two weeks validating X"*). The goal isn't perfect knowledge, but **enough confidence** to take the next step (*which might be more discovery or starting a small delivery iteration*). It's a continuous loop, not a one-off phase.

5. What are the most important KPIs I should track to understand if our development process (*SDLC*) is healthy and improving, without getting lost in technical details?

- **Focus on Flow & Outcomes:** Track metrics that reflect speed, stability, and value delivery. Key examples inspired by frameworks like DORA include:
 - **Lead Time for Changes:** How long does it take from a code commit to production? (*Measures speed/efficiency*).
 - **Deployment Frequency:** How often do you release to production? (*Measures throughput/agility*).

- **Change Failure Rate:** What percentage of deployments cause a failure in production? (*Measures quality/stability*).
- **Time to Restore Service:** How quickly can you recover from a production failure? (*Measures resilience*).
- **Keep it Simple & Track Trends:** Start with just a few key metrics. Focus on *trends* over absolute numbers. Are you getting faster, more stable, delivering value more reliably?
- **Connect to Business Goals:** Ultimately, link these technical KPIs back to business outcomes like revenue, user satisfaction, or market share.

6. Do we *really* need separate Product Managers and Project Managers? Can't one person do both, especially in a startup?

- **It's About Skills & Focus, Not Just Titles:** In very small teams (*1-4 people, often Type A*), one person **can** wear both hats. However, the **skillsets** are distinct:
 - **Product Manager:** Focuses on the **What** and **Why** (*vision, market needs, user value, prioritization based on business goals*).
 - **Project Manager:** Focuses on the **How** and **When** (*coordination, planning, removing roadblocks, managing risk, ensuring efficient execution*).
- **Separation Aids Scaling & Clarity:** As complexity grows (*more people, Type B projects*), separating these roles becomes crucial to avoid bottlenecks and ensure both strategic direction *and* effective delivery receive adequate attention. A dedicated Project Manager frees the Product Manager to focus on strategy and the Dev Team to focus on building.

7. How can I ensure transparency and alignment between business goals and technical execution, truly getting rid of the 'black box'?

- **Shared Understanding is Key:** Foster continuous collaboration. Involve tech leads early in Discovery. Use techniques like **OOPSI**, **User Story Mapping**, or **BDD** to create a **common language** and ensure everyone understands the **why** behind the **what**.
- **Make the Process Visible:** Implement a clear, agreed-upon SDLC (*like the one in this book*). Use visual tools (*Kanban boards, roadmaps*) to track progress. Ensure the backlog reflects clear priorities tied to business value.
- **Focus on Outcomes & Metrics:** Define success metrics upfront for features (*ROI, user adoption*). Regularly review progress against these outcomes, not just technical task completion. Use relevant KPIs (*see question 5*) to monitor process health.

- **Build Trust Through Process:** Transparency isn't about understanding every line of code; it's about having a trustworthy process, clear communication channels, and focusing dialogue on business value and risk mitigation.
-

A Flexible Framework, Not a Rigid Recipe

This framework isn't a **one-size-fits-all** solution, but rather an **adaptable model** you can tweak based on your team and your organization's reality. Every company is different, yet certain SDLC fundamentals—like continuous feedback loops, clear responsibilities, and iterative improvements—apply in nearly any context.

Self-Assessment: Where Is Your SDLC Today?

Use these questions to evaluate your current SDLC and spot improvement opportunities:

1. **Do you know if your company needs a Type A Project or a Type B Project?**
2. **Does your development team have crystal clarity on the problem they're solving?**
3. **Is there a defined process for validating assumptions before coding begins?**
4. **Are you measuring SDLC efficiency and predictability with clear KPIs?**
5. **Do your business and tech teams speak the same language, or is there constant friction?**
6. **How long does it take to go from idea to production for a new feature?**
7. **Does the team have structured processes for automation, testing, and deployment?**
8. **Are there mechanisms to regularly evaluate and refine the SDLC?**

If most of your answers are “**no**” or “**not sure**,” you've got a prime opportunity to improve development and make it more efficient and predictable.

Next Steps: Let's Work Together to Optimize Your SDLC

If this book helped you get a clearer picture of the SDLC, and you're ready to **take your development team to the next level**, I can provide more personalized support.

Schedule a session with me to analyze your SDLC and map out an action plan:
<https://calendly.com/pabloleone/lets-talk>

Get in touch if you're ready to apply this framework in your team:
<https://www.linkedin.com/in/pabloleone/>

Having a well-structured SDLC doesn't just boost team efficiency—it **reduces costs, speeds up software delivery**, and increases satisfaction for both customers and the development team.

Thanks for reading. Now's the time to put this framework into action and **optimize software development** in your organization!

About the Author

With over a decade of experience leading software teams across Europe, Pablo Leone has built scalable systems, coached engineers, and helped companies improve the way they deliver software. A self-taught developer turned Team Lead, his career blends hands-on engineering, technical leadership, and a deep interest in organizational design.

Driven by a passion for clarity, simplicity, and impact, he believes software development is not just about code—it's about aligning people, processes, and products to deliver real value. This book distills years of real-world lessons, failures, and breakthroughs into a practical framework for navigating the Software Development Lifecycle (*SDLC*) with purpose and efficiency.

When he's not writing or consulting, you'll find him training at the gym, playing the drums, or exploring the balance between work, family, and freedom.

Further Reading & Resources

To go deeper into the ideas explored in this book, here are the essential books that inspired and shaped this framework. Each of them brings a unique perspective—from technical practices and architectural clarity to team dynamics and product strategy. Together, they form a solid foundation for building effective teams and scalable software.

Product Discovery & Strategy

- **Inspired** – *Marty Cagan*
Learn how successful product teams build what users really need and avoid building what they don't.

- **Empowered** – *Marty Cagan*
Shows how strong product teams operate with autonomy and accountability, aligning closely with the Product Discovery side of this framework.
- **Impact Mapping** – *Gojko Adzic*
A visual planning technique to connect deliverables with real business goals—key for prioritizing the right experiments.
- **User Story Mapping** – *Jeff Patton*
Helps structure and prioritize the product backlog around actual user journeys, improving alignment and delivery focus.
- **BDD (Behavior-Driven Development)** – *Dan North*
Introduces a shared language for specifying behavior using "Given-When-Then", improving clarity across roles and reducing ambiguity.
- **Specification by Example** – *Gojko Adzic*
Encourages teams to turn requirements into concrete, testable examples—essential for collaborative validation.
- **OOPSI** – *Jenny Martin & Pete Buckney*
A facilitation framework to co-create shared understanding with stakeholders by breaking down features into **Outcomes, Outputs, Processes, Scenarios**, and **Inputs**.
- **50 Quick Ideas for Improving Your User Stories** – *Gojko Adzic & David Evans*
Offers practical patterns to make your backlog easier to manage, slice, and align with user goals.
- **Drive: The Surprising Truth About What Motivates Us** – *Daniel H. Pink*
Explores autonomy, mastery, and purpose as core drivers of motivation—crucial elements to consider when building empowered and high-performing teams.

Software Delivery & Engineering Excellence

- **Clean Code** – *Robert C. Martin*
A classic guide to writing code that's readable, testable, and maintainable—especially critical for Type B systems.
- **Clean Architecture** – *Robert C. Martin*
Focuses on separating concerns and building systems that are resilient to change.
- **Code Complete** – *Steve McConnell*
A deep dive into writing robust software, with practical techniques and insights into software construction.

- **Extreme Programming** – *Kent Beck*
Introduces practices like TDD, pair programming, and continuous feedback, many of which inspired parts of this framework.
- **Refactoring** – *Martin Fowler*
Teaches how to improve the design of existing code safely—vital when transitioning from experimental to production-grade systems.
- **Growing Object-Oriented Software Guided by Tests** – *Steve Freeman & Nat Pryce*
Provides a real-world approach to driving design through testing—bridging the gap between TDD and architecture.
- **50 Quick Ideas for Improving Your Tests** – *Gojko Adzic, David Evans & Tom Roden*
Helps strengthen your test strategy with fast, actionable ideas for making tests more reliable and maintainable.
- **50 Quick Ideas for Improving Your Retrospectives** – *Ben Williams & Tom Roden*
Practical suggestions to keep team retrospectives fresh, focused, and useful for continuous improvement.

Architecture & Scalability

- **Domain-Driven Design** – *Eric Evans*
A deep and structured way to model complex domains—crucial for long-lived systems that evolve over time.
- **Implementing Domain-Driven Design (*Red Book*)** – *Vaughn Vernon*
A practical take on applying DDD patterns and architecture in real-world applications.
- **Design Patterns** – *Gang of Four*
The classic reference for reusable solutions to common design problems, forming the backbone of many modern architectures.
- **Modern Software Engineering** – *Dave Farley*
Emphasizes the importance of continuous flow, feedback, and engineering discipline in modern delivery.
- **Release It!** – *Michael T. Nygard*
Focuses on the operational stability of software in production and teaches how to design systems that survive real-world failures.
- **Continuous Delivery** – *Jez Humble & David Farley*
Lays the groundwork for building deployment pipelines and reducing delivery risk—especially for complex systems.

DevOps & Team Dynamics

- **Accelerate** – *Nicole Forsgren, Jez Humble & Gene Kim*
Presents research-backed insights on what makes software teams high-performing—including key metrics like DORA.
- **The DevOps Handbook** – *Gene Kim, Jez Humble, Patrick Debois & John Willis*
Combines technical and cultural practices for breaking down silos and improving flow from idea to production.
- **Team Topologies** – *Matthew Skelton & Manuel Pais*
Provides a modern blueprint for organizing teams to reduce friction and improve delivery through clear interaction patterns.

Leadership, Culture & Career

- **Peopleware** – *Tom DeMarco & Tim Lister*
Argues that most software problems are people problems. Essential for understanding team productivity and culture.
- **The Clean Coder** – *Robert C. Martin*
Focuses on the discipline, ethics, and mindset of professional software development.
- **The Effective Engineer** – *Edmond Lau*
Offers strategies to maximize impact and focus your time where it matters most in engineering work.
- **Soft Skills** – *John Sonmez*
Covers personal branding, productivity, and career growth for software developers beyond code.
- **Crucial Conversations** – *Kerry Patterson, Joseph Grenny & co.*
Provides tools for handling difficult conversations with clarity and confidence—vital for team leads and managers.
- **The Complete Software Developer's Career Guide** – *John Sonmez*
A broad roadmap for developers at any stage, covering technical growth, mindset, and long-term career strategies in software.
- **High Output Management** – *Andrew Grove*
A masterclass in managing teams, setting up systems, and driving productivity. Essential reading for any engineering or product leader.
- **The Manager's Path: A Guide for Tech Leaders Navigating Growth and Change** – *Camille Fournier*

An essential guide for engineers transitioning into leadership roles. It covers every stage—from mentoring interns to becoming a CTO—offering actionable insights into managing people, processes, and your own growth as a leader.

These books complement the ideas in this book and will help you expand your understanding of what makes modern software development truly effective—from individual contributors to multi-team organizations.

Do you feel like software development in your company is a “black box” filled with chaos, delays, and unexpected costs?

Are you frustrated by your inability to make strategic decisions around technology because you lack a technical background?

Software Development Without Chaos is a practical guide written specifically for non-technical CEOs, founders, and business leaders like you.

Inside, you'll learn how to:

- Eliminate the “Black Box”: Gain visibility and control over the entire development process—from idea to delivery.
- Make Strategic Decisions with Confidence: Learn to distinguish when to run a fast experiment (Type A / MVP) and when to invest in a scalable, robust system (Type B), aligning technology with business goals.
- Achieve Predictable Results: Implement lightweight processes and metrics that reduce uncertainty and ensure consistent value delivery.
- Improve Communication: Foster effective collaboration between business and tech teams, avoiding costly misunderstandings.
- Manage Risk: Understand and mitigate common issues like technical debt and team misalignment.

Based on over a decade of real-world experience leading software teams, Pablo Leone distills practical lessons into a method you can apply immediately.

Stop feeling at the mercy of technical complexity—and start leading software development with confidence and strategic clarity.

